실무 이펙티브 자바 Day 1

객체지향 설계의 본질과 안전한 도메인 모델링 (Java 8)

생존코딩 오준석

이틀 동안 SCV 한 기를 계속 고친다

오늘은 SCV와 테란 유닛 코드를 고쳐 가며 문제 여덟 개를 만난다. 지금은 질문만 보여 준다.

교시	오늘 만날 문제
1	SCV는 생체인가, 기계인가?
2	체력 계산식이 다섯 군데에 있다면?
3	체력 1인 SCV는 어떻게 시험하나?
4	집결지를 옮겼더니 SCV가 순간이동했다?
5	생성자에 넘긴 숫자 일곱 개는 각각 무엇인가?
6	자원을 닫다가 원래 예외가 사라졌다?
7	같은 좌표인데 HashSet에서 못 찾는다?
8	도서 목록은 어떤 순서로 정렬해야 하나?

이 과정의 약속: Java 8, 스프링 없이, 테스트로 확인

쓰는 것	쓰지 않는 것
Java 8 문법과 JDK 표준 라이브러리	`var`, `record`, `List.of` 같은 Java 9 이후 문법
클래스, 인터페이스, 생성자를 직접 작성	Spring, Lombok, Guava
JUnit 5 테스트(`@Test`, `@DisplayName`)	Mockito, AssertJ, JUnit 4

현업 시스템이 Java 8이라서 오늘 배운 코드를 내일 회사에서 그대로 쓸 수 있게 한다.

> 스프링을 빼는 이유: 프레임워크가 대신 정해 주던 설계를 언어만으로 직접 결정해 보기 위해서다.

확인은 JUnit 5 테스트로: 실패 → 수정 → 통과

모든 개념은 빨간 테스트로 시작해서, 코드를 고친 뒤 초록 테스트로 끝난다. 이 장은 형식만 본다(따라 치는 것은 SCV를 만든 뒤).

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class ScvStartTest {

    @Test
    @DisplayName("새로 만든 SCV의 체력은 60이다")
    void newScvHasFullHp() {
        SCV scv = new SCV();

        assertEquals(60, scv.getHp());
    }
}
```

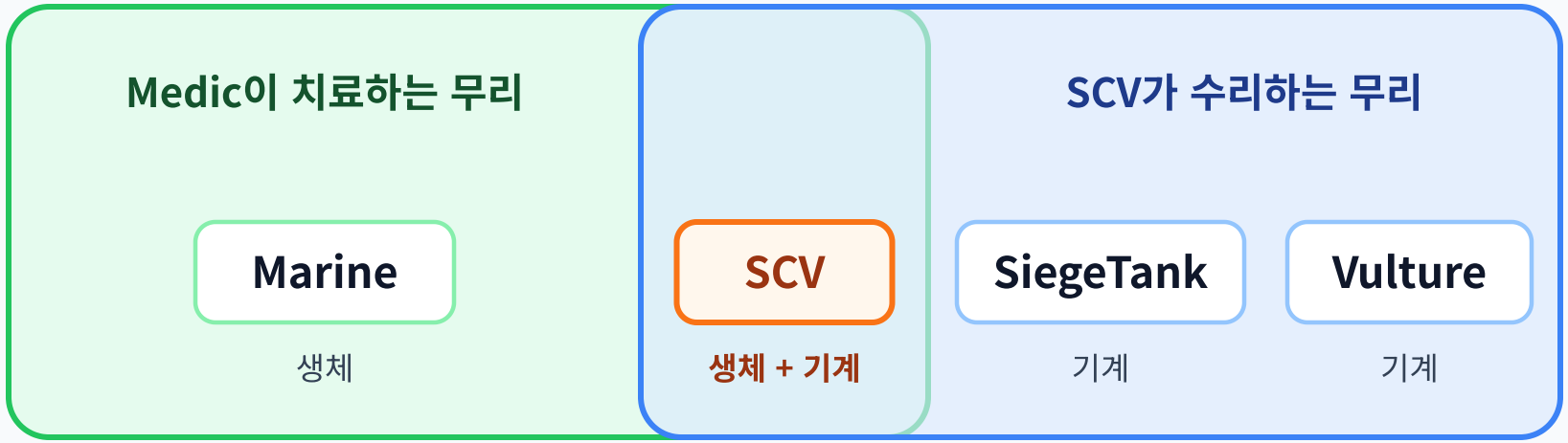
> JUnit 4와 다른 점은 `@Test`의 패키지(`org.junit.jupiter.api`)와 `@DisplayName` 정도다. 테스트 클래스와 메서드에는 `public`을 붙이지 않는다.

1교시

SCV는 생체인가, 기계인가

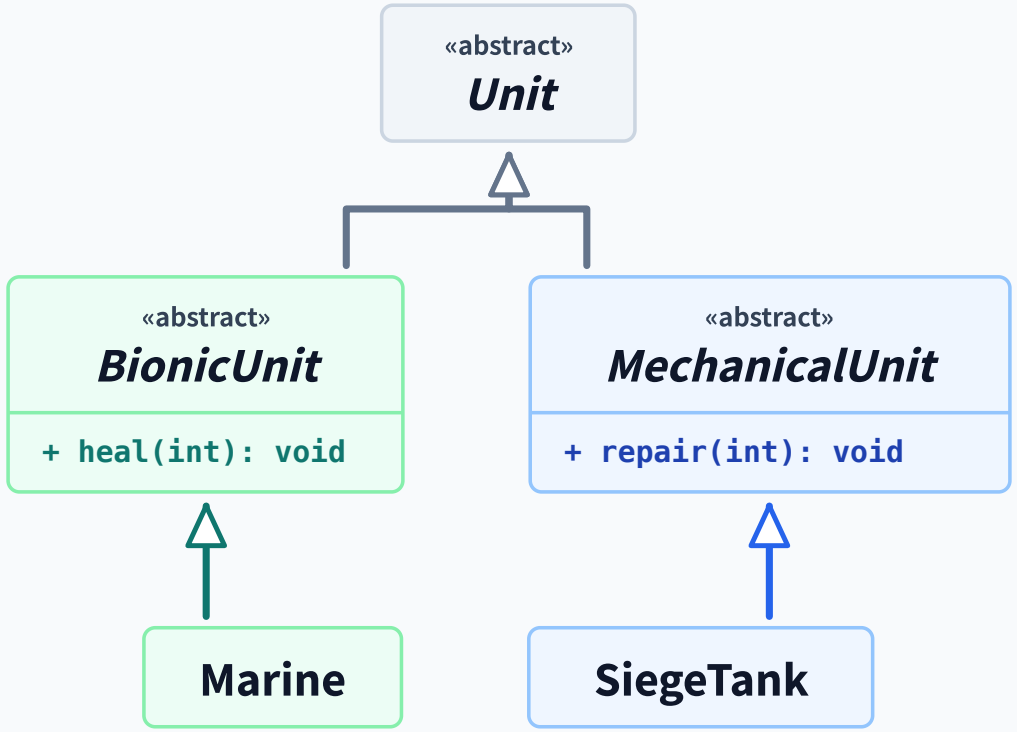
치료도 받고 수리도 받는 유닛을 코드로 옮겨 본다.

메딕은 생체 유닛을 치료하고, SCV는 기계 유닛을 수리한다



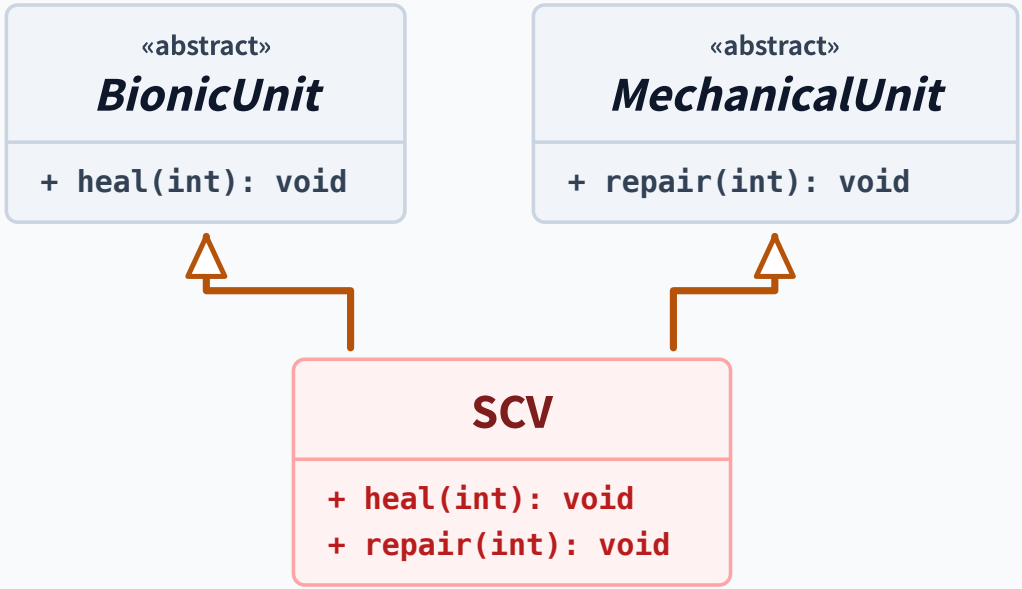
마린은 메딕이 치료하고, 시즈 탱크와 벌처는 SCV가 수리한다. 그런데 SCV는 사람이 타는 기계라서 **메딕에게 치료도, 다른 SCV에게 수리도** 받아야 한다.

상속으로 나눠 본다: Unit → BionicUnit / MechanicalUnit



화살표는 자식에서 부모로 간다(물려받는다). 생체 부모는 `heal`, 기계 부모는 `repair`를 약속한다.

SCV는 두 부모가 모두 필요하다




```
public class SCV extends BionicUnit, MechanicalUnit { }  // 컴파일 에러: '{' expected
```

자바 클래스는 부모를 **하나만** 가질 수 있다. 둘 중 하나를 포기하는 편법이 필요하다.

편법 1 준비: Unit과 두 부모 클래스

그림의 계층을 그대로 옮긴다. 클래스마다 파일 하나를 만든다.

Unit.java

```
public abstract class Unit {
    protected int hp;

    public int getHp() {
        return hp;
    }
}
```

BionicUnit.java

```
public abstract class BionicUnit extends Unit {
    public abstract void heal(int amount);
}
```

MechanicalUnit.java

```
public abstract class MechanicalUnit extends Unit {
    public abstract void repair(int amount);
}
```

편법 1 준비: Marine과 SiegeTank

체력은 각 유닛 생성자에서 최대치로 채운다.

Marine.java

```java
public class Marine extends BionicUnit {
    private static final int MAX_HP = 40;

    public Marine() {
        hp = MAX_HP;
    }

    @Override
    public void heal(int amount) {
        hp = Math.min(hp + amount, MAX_HP);
    }
}
```

SiegeTank.java

```java
public class SiegeTank extends MechanicalUnit {
    private static final int MAX_HP = 150;

    public SiegeTank() {
        hp = MAX_HP;
    }

    @Override
    public void repair(int amount) {
        hp = Math.min(hp + amount, MAX_HP);
    }
}
```

편법 1: BionicUnit을 상속하고 repair를 복사한다

생체 쪽 부모를 고르고, 수리 코드는 `SiegeTank`에서 그대로 복사한다.

SCV.java

```
public class SCV extends BionicUnit {
    private static final int MAX_HP = 60;
    static final int REPAIR_AMOUNT = 10;

    public SCV() {
        hp = MAX_HP;
    }

    @Override
    public void heal(int amount) {
        hp = Math.min(hp + amount, MAX_HP);
    }

    // SiegeTank.repair에서 복사
    public void repair(int amount) {
        hp = Math.min(hp + amount, MAX_HP);
    }

    public void repairTarget(MechanicalUnit target) {
        target.repair(REPAIR_AMOUNT);
    }
}
```

Medic.java

```
public class Medic {
    static final int HEAL_AMOUNT = 10;

    public void treat(BionicUnit target) {
        target.heal(HEAL_AMOUNT);
    }
}
```

`repair` 메서드는 생겼다. 이제 SCV는 수리를 받을 수 있을까?

편법 1의 대가: SCV를 repairTarget(MechanicalUnit)에 넘길 수 없다

수리하는 쪽 메서드는 기계 유닛만 받는다. 발췌(앞 장 `SCV.java`):

```
public void repairTarget(MechanicalUnit target) {  // SCV 안
    target.repair(REPAIR_AMOUNT);
}
```

```
scv.repairTarget(tank);      // 된다
scv.repairTarget(otherScv);  // 컴파일 에러: SCV cannot be converted to MechanicalUnit
```

SCV에는 `repair` 메서드가 **있다**. 하지만 타입은 `MechanicalUnit`이 아니라서 수리 대상으로 받아 주지 않는다.

편법 2: 맨 위 Unit에 heal과 repair를 다 넣는다

부모를 하나로 합치고, 두 동작을 모두 둔다. 기본 구현은 아무 일도 하지 않는다.

Unit.java

```
// 편법 2: 맨 위 Unit에 heal과 repair를 다 넣고, 기본은 아무 일도 하지 않는다
public abstract class Unit {
    protected int hp;

    public void heal(int amount) {
    }

    public void repair(int amount) {
    }

    public int getHp() {
        return hp;
    }
}
```

1. `BionicUnit.java`, `MechanicalUnit.java`를 지운다.
2. `Marine.java`와 `SiegeTank.java`는 `extends` 뒤만 `Unit`으로 바꾼다.
3. `Medic.java`의 `treat`를 이것으로 바꾼다.

```
public void treat(Unit target) {
    target.heal(HEAL_AMOUNT);
}
```

편법 2: SCV는 Unit 하나만 상속하고 둘 다 재정의한다

`SCV.java` 를 이것으로 바꾼다. 이제 SCV는 어디에든 넘길 수 있다.

```
public class SCV extends Unit {
    private static final int MAX_HP = 60;
    static final int REPAIR_AMOUNT = 10;

    public SCV() {
        hp = MAX_HP;
    }

    @Override
    public void heal(int amount) {
        hp = Math.min(hp + amount, MAX_HP);
    }

    @Override
    public void repair(int amount) {
        hp = Math.min(hp + amount, MAX_HP);
    }

    public void repairTarget(Unit target) {
        target.repair(REPAIR_AMOUNT);
    }
}
```

편법 2의 대가: 메딕이 시즈 탱크를 치료해도 컴파일된다

MedicTreatTest.java `Medic.treat(Unit target)`은 모든 `Unit`을 받는다. 이 테스트는 **통과한다.**

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

// 편법 2의 문제를 기록하는 테스트: 잘못된 호출이 컴파일도 실행도 통과한다
class MedicTreatTest {

    @Test
    @DisplayName("메딕이 시즈 탱크를 치료해도 컴파일도 실행도 통과한다")
    void medicTreatsSiegeTankSilently() {
        Medic medic = new Medic();
        SiegeTank tank = new SiegeTank();

        medic.treat(tank);  // 잘못된 호출인데 아무도 막지 않는다

        assertEquals(150, tank.getHp());
    }
```

```java
    @Test
    @DisplayName("SCV가 마린을 수리해도 컴파일도 실행도 통과한다")
    void scvRepairsMarineSilently() {
        SCV scv = new SCV();
        Marine marine = new Marine();

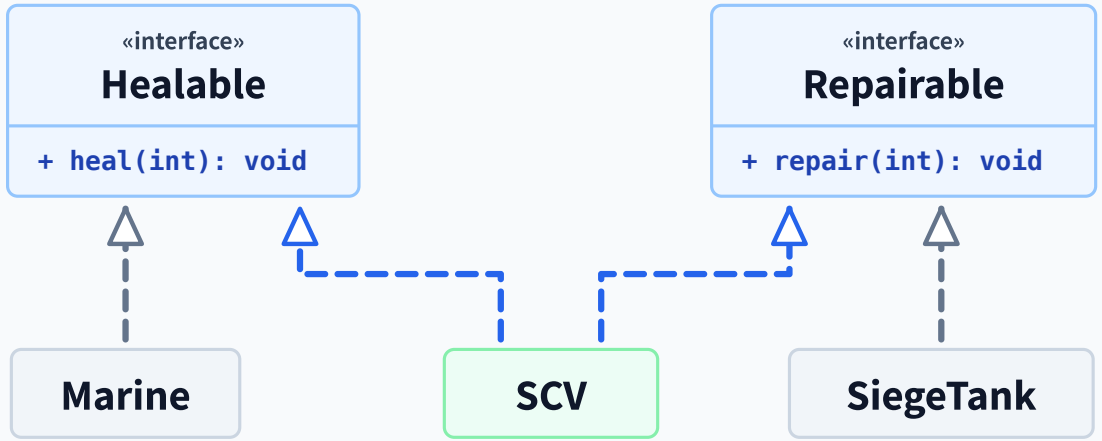
        scv.repairTarget(marine);  // 이것도 막지 않는다

        assertEquals(40, marine.getHp());
    }
}
```

실수를 컴파일러도 테스트도 잡지 못한다.

> 현업 장면: `AbstractXxxService`를 상속하다 두 번째 부모가 필요해 막혔던 경험이 바로 편법 1·2다.

신분 대신 역할을 타입으로 만든다



"생체냐 기계냐"(신분)가 아니라 "치료받을 수 있다", "수리받을 수 있다"(역할)를 인터페이스로 만든다. 클래스는 역할을 **여러 개** 구현할 수 있다.

> 이것이 이펙티브 자바 아이템 20: 추상 클래스보다는 인터페이스를 우선하라. SOLID로 보면 **ISP(인터페이스 분리 원칙)**다. 시즈 탱크가 쓰지도 않는 `heal`에 묶이지 않도록 역할을 잘게 나눈다.

Healable, Repairable과 SCV 최소 구현

Healable.java

```java
public interface Healable {
    void heal(int amount);
}
```

Repairable.java

```java
public interface Repairable {
    void repair(int amount);
}
```

1. 역할 인터페이스 두 개를 만든다.
2. `SCV.java`를 오른쪽 코드로 바꾼다. 부모 클래스가 없으니 `hp`와 `getHp()`를 직접 가진다.
3. `Unit.java`는 지운다.

SCV.java

```java
public class SCV implements Healable, Repairable {
    private static final int MAX_HP = 60;
    static final int REPAIR_AMOUNT = 10;

    private int hp = MAX_HP;

    @Override
    public void heal(int amount) {
        hp = Math.min(hp + amount, MAX_HP);
    }

    @Override
    public void repair(int amount) {
        hp = Math.min(hp + amount, MAX_HP);
    }

    public void repairTarget(Repairable target) {
        target.repair(REPAIR_AMOUNT);
    }

    public int getHp() {
        return hp;
    }
}
```

받는 쪽은 역할만 본다

`Medic.treat`와 `SCV.repairTarget`(앞 장)의 매개변수 타입이 클래스 계층 대신 역할 인터페이스다.

Medic.java

```
public class Medic {
    static final int HEAL_AMOUNT = 10;

    public void treat(Healable target) {
        target.heal(HEAL_AMOUNT);
    }
}
```

```
medic.treat(scv);            // SCV는 Healable
scv.repairTarget(otherScv);  // SCV는 Repairable: 편법 1에서 막혔던 호출
medic.treat(tank);           // 컴파일 에러: SiegeTank cannot be converted to Healable
```

편법 2에서 통과하던 잘못된 호출은 이제 컴파일러가 막는다. `MedicTreatTest.java`가 컴파일되지 않는 것을 확인하고 지운다.

Marine과 SiegeTank도 자기 역할만 구현한다

두 파일을 이것으로 바꾼다. 마린은 치료만, 시즈 탱크는 수리만 받는다.

Marine.java

```java
public class Marine implements Healable {
    private static final int MAX_HP = 40;

    private int hp = MAX_HP;

    @Override
    public void heal(int amount) {
        hp = Math.min(hp + amount, MAX_HP);
    }

    public int getHp() {
        return hp;
    }
}
```

SiegeTank.java

```java
public class SiegeTank implements Repairable {
    private static final int MAX_HP = 150;

    private int hp = MAX_HP;

    @Override
    public void repair(int amount) {
        hp = Math.min(hp + amount, MAX_HP);
    }

    public int getHp() {
        return hp;
    }
}
```

테스트로 확인: 치료도 수리도 최대 체력을 넘지 않는다

ScvRoleTest.java 앞에서 형식만 본 ScvStartTest.java 도 이제 붙여 넣는다.

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class ScvRoleTest {

    @Test
    @DisplayName("메딕이 SCV를 치료해도 최대 체력 60을 넘지 않는다")
    void medicHealsScvUpToMaxHp() {
        Medic medic = new Medic();
        SCV scv = new SCV();

        medic.treat(scv);

        assertEquals(60, scv.getHp());
    }

    @Test
    @DisplayName("SCV가 다른 SCV를 수리해도 최대 체력 60을 넘지 않는다")
    void scvRepairsAnotherScvUpToMaxHp() {
        SCV repairer = new SCV();
        SCV target = new SCV();

        repairer.repairTarget(target);

        assertEquals(60, target.getHp());
    }

    @Test
    @DisplayName("SCV가 시즈 탱크를 수리해도 최대 체력 150을 넘지 않는다")
    void scvRepairsSiegeTankUpToMaxHp() {
        SCV scv = new SCV();
        SiegeTank tank = new SiegeTank();

        scv.repairTarget(tank);

        assertEquals(150, tank.getHp());
    }
}
```

두 파일 4건 모두 통과한다. SCV 하나가 두 역할로 다 받아들여진다.

벌처를 수리 대상에 넣는다 (실습)

실습 파일: `Vulture.java` (시작 코드)

```
public class Vulture {
    private static final int MAX_HP = 80;

    private int hp = MAX_HP;

    public void repair(int amount) {
        throw new UnsupportedOperationException("실습: repair");
    }

    public int getHp() {
        return hp;
    }
}
```

1. `Vulture`가 `Repairable` 역할을 선언한다.
2. `repair`가 체력을 80 넘게 올리지 않게 구현한다.

왜: 이미 있는 클래스에 역할 하나를 더 붙이는 일이 얼마나 작은지 직접 확인한다.

실습 테스트: VultureTest 2건을 초록으로

VultureTest.java 지금은 2건 모두 빨간색이고, 바꾼 뒤에는 2건 모두 초록이어야 한다.

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class VultureTest {

    @Test
    @DisplayName("벌처는 수리 대상이다")
    void vultureIsRepairable() {
        Vulture vulture = new Vulture();

        assertTrue(vulture instanceof Repairable);
    }
```

```java
    @Test
    @DisplayName("수리를 받아도 벌처 체력은 80을 넘지 않는다")
    void repairDoesNotExceedMaxHp() {
        Vulture vulture = new Vulture();

        vulture.repair(SCV.REPAIR_AMOUNT);

        assertEquals(80, vulture.getHp());
    }
}
```

실습 풀이: repair 메서드만으로는 수리 대상이 아니다

시작 상태의 실패 메시지:

```
벌처는 수리 대상이다 — expected: <true> but was: <false>
수리를 받아도 벌처 체력은 80을 넘지 않는다 — UnsupportedOperationException: 실습: repair
```

Vulture.java

```java
public class Vulture implements Repairable {
    private static final int MAX_HP = 80;

    private int hp = MAX_HP;

    @Override
    public void repair(int amount) {
        hp = Math.min(hp + amount, MAX_HP);
    }

    public int getHp() {
        return hp;
    }
}
```

`VultureTest` 2건 통과. 시작 코드에도 `repair`가 있었지만 `implements Repairable`을 선언하기 전에는 수리 대상이 아니었다. 역할은 선언 한 줄로 붙는다.

Java 8에 default 메서드가 생겼으니 추상 클래스는 이제 필요 없을까? (퀴즈)

퀴즈

인터페이스에도 `default` 메서드로 본문을 쓸 수 있게 되었다. 그렇다면?

1. 필요 없다. `default` 메서드로 추상 클래스가 하던 일을 모두 할 수 있다.
2. 필요하다. 인터페이스는 인스턴스 필드(상태)를 가질 수 없다.
3. 필요하다. 인터페이스 메서드는 본문을 가질 수 없다.

정답: 인터페이스는 상태를 가질 수 없다

정답

정답은 2번이다. 인터페이스에 선언한 필드는 모두 `public static final` 상수가 된다. 유닛마다 다른 `hp`는 담을 수 없다. 3번은 `default` 메서드가 있으니 틀렸다.

- 그래서 `hp`와 `Math.min(hp + amount, MAX_HP)`는 `SCV`, `Marine`, `SiegeTank`에 각각 남아 있다.
- 이미 배포한 인터페이스에 `default` 메서드를 추가하면, 같은 이름의 메서드를 가진 기존 구현 클래스와 충돌할 수 있다.

> 역할은 인터페이스로 나눴다. 상태를 가진 공통 구현은 어디에 둘까? 다음 시간의 출발점이다.

2교시

체력 계산식이 다섯 군데에

같은 한 줄이 여러 클래스에 복사돼 있다. 규칙이 바뀌는 날 무슨 일이 생길까?

Marine.heal, SCV.heal, SCV.repair에 같은 Math.min

발췌: 1교시 끝의 코드에서 체력을 올리는 메서드만 뽑았다(필드와 `@Override` 생략). 역할은 인터페이스로 나눴지만, 체력을 올리는 한 줄은 클래스마다 있다.

```
public class Marine implements Healable {
    public void heal(int amount) {
        hp = Math.min(hp + amount, MAX_HP);   // ①
    }
}

public class SCV implements Healable, Repairable {
    public void heal(int amount) {
        hp = Math.min(hp + amount, MAX_HP);   // ②
    }

    public void repair(int amount) {
        hp = Math.min(hp + amount, MAX_HP);   // ③
    }
}
```

`SiegeTank.repair`, `Vulture.repair`까지 세면 같은 줄이 **다섯 곳**에 있다.

규칙 하나가 바뀌면 다섯 곳을 고친다

버그 리포트: 회복량에 `-5`가 들어오면 체력이 오히려 깎인다. 새 규칙은 「음수 회복량은 거부한다」.

#	검사를 넣어야 하는 곳
①	`Marine.heal`
②	`SCV.heal`
③	`SCV.repair`
④	`SiegeTank.repair`
⑤	`Vulture.repair`

> 하나만 빠뜨려도 그 유닛만 음수 회복을 받아들인다. 확인할 테스트도 다섯 벌이다. 이 계산을 한 곳에 모을 수 없을까?

default 메서드로 올리면? 인터페이스는 hp를 가질 수 없다

1교시 퀴즈에서 본 대로, 인터페이스에 쓴 필드는 상수가 된다. 그래도 `Healable`에 올려 보면 이렇게 된다.

```
public interface Healable {
    int hp = 60;   // 필드처럼 보이지만 public static final 상수

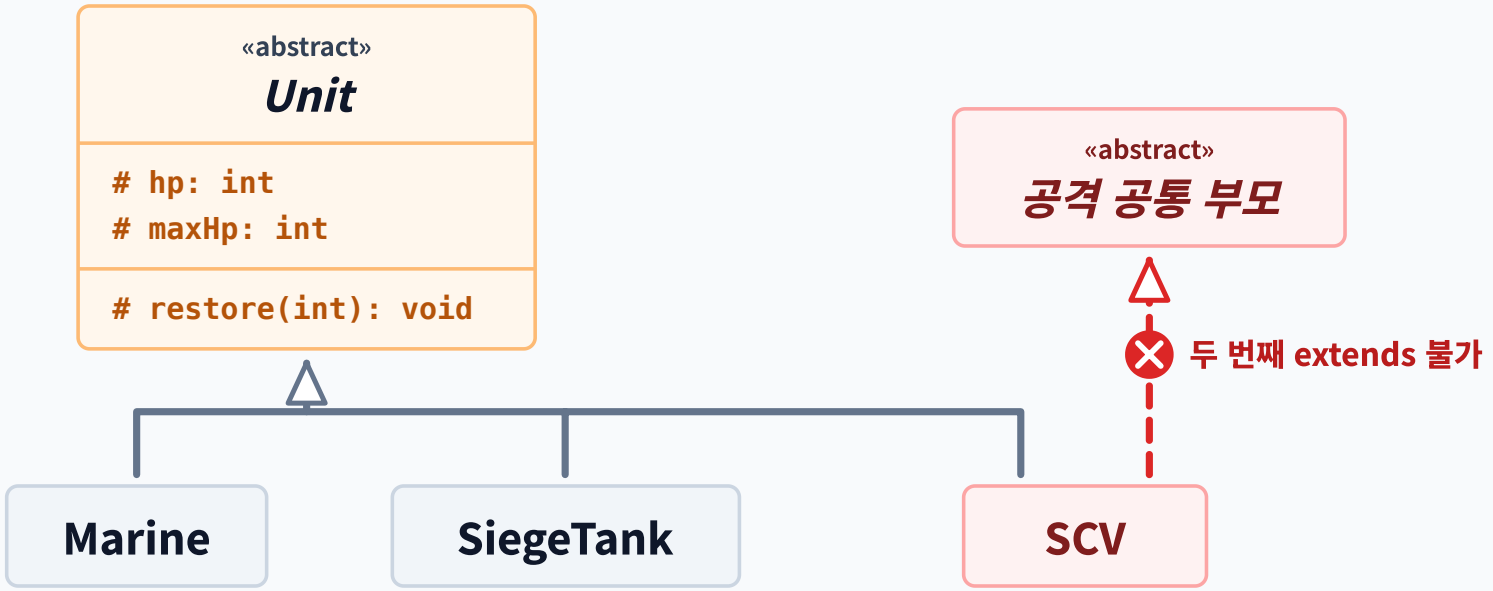
    default void heal(int amount) {
        hp = Math.min(hp + amount, 60);   // 컴파일 에러
    }
}
```

```
error: cannot assign a value to static final variable hp
```

JDK 버전에 따라 문구가 조금 다르다(JDK 8은 `static` 없이 `final variable hp`로 나올 수 있다).

유닛마다 다른 체력을 담을 곳이 없다. default 메서드에는 상태가 없는 공통 동작만 넣을 수 있다.

부모 클래스로 올리면? 부모 자리 하나를 체력이 차지한다



`hp`와 `restore`를 추상 클래스 `Unit`에 두면 중복은 사라진다. 대신 하나뿐인 부모 자리를 체력이 차지한다. `Unit`을 상속하면 다른 부모는 상속할 수 없고, 공격 코드를 별도 부모에 두어도 같은 문제가 생긴다.

HashSet의 addAll은 내부에서 add를 다시 부른다

CountingUnitSet.java 생산된 유닛 이름을 모으면서 몇 번 추가했는지 센다. 저장은 JDK `HashSet`을 물려받는다.

```java
import java.util.Collection;
import java.util.HashSet;

public class CountingUnitSet extends HashSet<String> {
    private static final long serialVersionUID = 1L;

    private int addCount = 0;

    @Override
    public boolean add(String unitName) {
        addCount++;
        return super.add(unitName);
    }

    @Override
    public boolean addAll(Collection<? extends String> unitNames) {
        addCount += unitNames.size();
        return super.addAll(unitNames);
    }

    public int getAddCount() {
        return addCount;
    }
}
```

세 개를 addAll 했는데 카운트는 6

CountingUnitSetTest.java

```java
import java.util.Arrays;

import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class CountingUnitSetTest {

    @Test
    @DisplayName("유닛 이름 세 개를 addAll 하면 추가 횟수는 3이다")
    void addAllCountsEachNameOnce() {
        CountingUnitSet names = new CountingUnitSet();

        names.addAll(Arrays.asList("SCV", "Marine", "SiegeTank"));

        assertEquals(3, names.getAddCount());
    }
}
```

실행 결과:

```
expected: <3> but was: <6>
```

부모의 `addAll`이 재정의한 `add`를 다시 불러 카운트가 6이 된다.

> 현업 장면: 공통 로직을 `BaseService`에 몰았다가 하위 서비스가 부모 내부 구현에 묶인 경험이 이것이다.

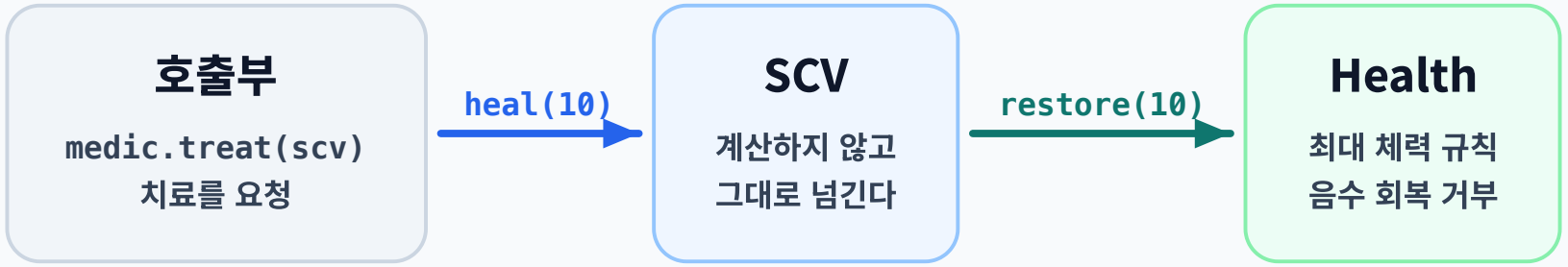
체력은 따로 떼어 '가진다'



체력 상태와 회복 규칙을 `Health` 라는 별도 객체로 떼어 낸다. `SCV` 와 `Marine` 은 `Health` 를 물려받지 않고 **필드로 가진다**. 실선 화살표는 "필드로 들고 있다"는 뜻이다. 역할 인터페이스 `Healable` , `Repairable` 은 그대로 구현한다.

> 이것이 이펙티브 자바 아이템 18: 상속보다는 컴포지션을 사용하라. 다른 객체를 필드로 포함해 쓰는 방식을 컴포지션(구성)이라 한다. SOLID로 보면 **SRP(단일 책임 원칙)**다. 체력 규칙이 바뀌면 `Health` 하나만 고친다.

받은 일을 넘긴다: 위임



SCV는 치료 요청을 받으면 직접 계산하지 않고 자기 `Health`에게 그대로 넘긴다. 이것을 **위임**이라 한다. 규칙은 `Health`에만 있고, SCV는 요청이 지나가는 창구다.

> 비유: 창구 직원은 환불 금액을 직접 계산하지 않고 정산 담당에게 넘긴다. 계산 규칙이 바뀌어도 창구는 그대로다.

Health 최소 구현: 범위를 지키는 체력

Health.java 새 파일. 만들 때 범위를 검사하고, 회복할 때 음수와 상한을 검사한다.

```
public class Health {
    private final int maxHp;
    private int currentHp;

    public Health(int maxHp) {
        this(maxHp, maxHp);
    }

    public Health(int currentHp, int maxHp) {
        if (maxHp <= 0 || currentHp < 0 || currentHp > maxHp) {
            throw new IllegalArgumentException("올바르지 않은 체력 값입니다.");
        }
        this.currentHp = currentHp;
        this.maxHp = maxHp;
    }
```

```
    public void restore(int amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("회복량은 0 이상이어야 합니다.");
        }
        currentHp = Math.min(currentHp + amount, maxHp);
    }

    public int getHp() {
        return currentHp;
    }

    public int getMaxHp() {
        return maxHp;
    }
}
```

SCV는 Health에게 위임한다

1교시의 `SCV.java`를 고친다. 바뀐 곳이 여러 군데라 파일 전체를 이것으로 바꾼다.

```
public class SCV implements Healable, Repairable {
    private static final int MAX_HP = 60;
    static final int REPAIR_AMOUNT = 10;

    private final Health health = new Health(MAX_HP);

    @Override
    public void heal(int amount) {
        health.restore(amount);
    }

    @Override
    public void repair(int amount) {
        health.restore(amount);
    }

    public void repairTarget(Repairable target) {
        target.repair(REPAIR_AMOUNT);
    }

    public int getHp() {
        return health.getHp();
    }
}
```

`heal`과 `repair`는 한 줄짜리 전달이 됐다. 계산식은 SCV 안에 없다.

Marine도 같은 Health를 쓴다: 규칙은 한 곳에

`Marine.java` 를 이것으로 바꾼다.

```java
public class Marine implements Healable {
    private static final int MAX_HP = 40;

    private final Health health = new Health(MAX_HP);

    @Override
    public void heal(int amount) {
        health.restore(amount);
    }

    public int getHp() {
        return health.getHp();
    }
}
```

`MarineHealTest.java` 새 테스트

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class MarineHealTest {

    @Test
    @DisplayName("메딕이 마린을 치료해도 최대 체력 40을 넘지 않는다")
    void medicHealsMarineUpToMaxHp() {
        Medic medic = new Medic();
        Marine marine = new Marine();

        medic.treat(marine);

        assertEquals(40, marine.getHp());
    }
}
```

규칙	전	후
최대 체력 상한	다섯 메서드에 복사	`Health.restore` 한 곳
음수 회복량 거부	다섯 곳에 추가해야 함	`Health.restore`에 이미 있음

SiegeTank와 Vulture도 같은 모양으로 바꾼다

두 파일을 이것으로 바꾼다. 바뀌는 곳은 `Health` 필드, `repair`의 한 줄, `getHp`의 한 줄이다.

SiegeTank.java

```
public class SiegeTank implements Repairable {
    private static final int MAX_HP = 150;

    private final Health health = new Health(MAX_HP);

    @Override
    public void repair(int amount) {
        health.restore(amount);
    }

    public int getHp() {
        return health.getHp();
    }
}
```

Vulture.java

```
public class Vulture implements Repairable {
    private static final int MAX_HP = 80;

    private final Health health = new Health(MAX_HP);

    @Override
    public void repair(int amount) {
        health.restore(amount);
    }

    public int getHp() {
        return health.getHp();
    }
}
```

이제 `Math.min`은 `Health.restore` 안에 하나만 남는다.

테스트로 확인: 회복량과 회복 상한

HealthTest.java

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class HealthTest {

    @Test
    @DisplayName("체력 30에서 10 회복하면 40이 된다")
    void restoreAddsAmount() {
        Health health = new Health(30, 60);

        health.restore(10);

        assertEquals(40, health.getHp());
    }

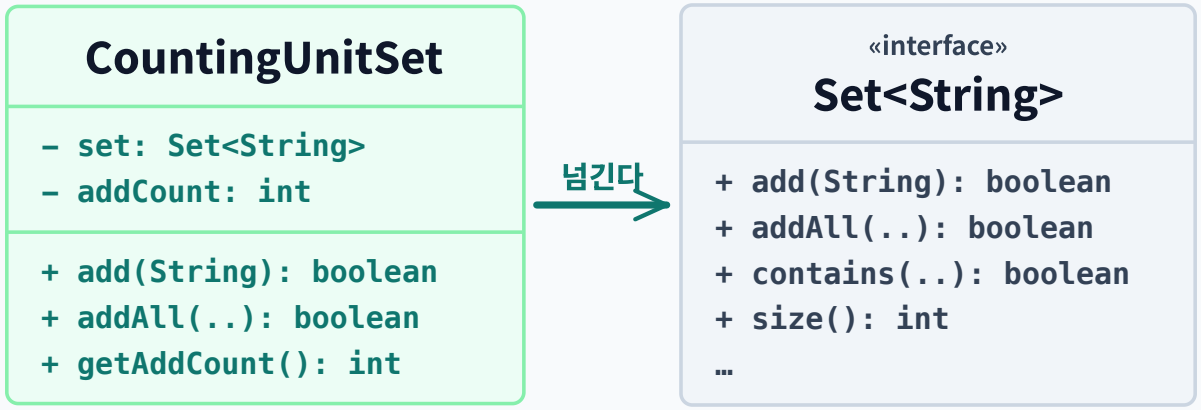
    @Test
    @DisplayName("체력 55에서 10 회복해도 최대 체력 60에서 멈춘다")
    void restoreStopsAtMaxHp() {
        Health health = new Health(55, 60);

        health.restore(10);

        assertEquals(60, health.getHp());
    }
}
```

`HealthTest` 2건, `MarineHealTest` 1건이 통과한다. 1교시의 `ScvStartTest`, `ScvRoleTest`, `VultureTest`도 고치지 않고 그대로 통과한다. 밖에서 보는 동작은 같고 안만 바뀌었다.

CountingUnitSet도 상속 대신 포함으로



내부 `Set`에 저장을 맡기고 필요한 메서드만 제공한다.

테스트 통과: addAll 세 개에 카운트 3

앞에서 만든 CountingUnitSet.java 를 고친다. `extends` 가 빠지고 필드가 생겨서 파일 전체를 바꾼다.

```
import java.util.Collection;
import java.util.HashSet;
import java.util.Set;

public class CountingUnitSet {
    private final Set<String> set = new HashSet<>();
    private int addCount = 0;

    public boolean add(String unitName) {
        addCount++;
        return set.add(unitName);
    }

    public boolean addAll(Collection<? extends String> unitNames) {
        addCount += unitNames.size();
        return set.addAll(unitNames);
    }

    public int getAddCount() {
        return addCount;
    }
}
```

내부 `HashSet` 의 `addAll` 은 바깥 `add` 를 다시 부르지 않아 중복 집계하지 않는다.

> 같은 `CountingUnitSetTest` 를 고치지 않고 다시 실행하면 이제 통과한다: 카운트 3.

상속을 써도 되는 때는 언제일까? (퀴즈)

퀴즈

1. 코드를 재사용하고 싶을 때는 언제나
2. 부모 클래스가 JDK처럼 믿을 만한 라이브러리일 때
3. 두 클래스가 진짜 is-a 관계이고, 부모가 상속을 염두에 두고 설계·문서화됐을 때
4. 상속은 어떤 경우에도 쓰지 않는다

정답: 진짜 is-a이고, 부모가 확장을 위해 설계됐을 때

정답

정답은 3번이다.

- 1번: 재사용만이 목적이면 포함·위임으로 충분하다. 오늘 본 `Health`가 그 예다.
- 2번: `HashSet`은 JDK지만 `addAll`이 `add`를 부른다는 내부 사정 때문에 카운트가 틀렸다.
- 4번: `AbstractList`처럼 확장용으로 만들고 재정의 규칙을 문서에 적은 클래스는 상속해도 된다. 같은 팀이 부모와 자식을 함께 관리하는 같은 패키지 안의 상속도 비교적 안전하다.

> 판단 질문: "B는 정말 A인가?" 그리고 "A는 내가 물려받아도 되게 만들어졌나?" 둘 다 예일 때만 `extends`를 쓴다.

Health에 damage를 추가한다 (실습)

실습 파일 수정: 앞에서 만든 `Health.java` · 테스트: `HealthDamageTest.java` (다음 장)

먼저 `Health.java`의 `restore` 아래에 이 시작 메서드를 넣는다.

```
public void damage(int amount) {
    throw new UnsupportedOperationException("실습: damage");
}
```

1. `damage`가 현재 체력을 `amount`만큼 줄이게 한다.
2. 체력은 0 아래로 내려가지 않는다.

왜: 피해 규칙도 `Health` 한 곳에만 생긴다. 유닛 클래스는 하나도 고치지 않는다.

실습 테스트: HealthDamageTest 2건을 초록으로

HealthDamageTest.java 지금은 2건 모두 빨간색이고, 바꾼 뒤에는 2건 모두 초록이어야 한다.

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class HealthDamageTest {

    @Test
    @DisplayName("체력 30에서 10 피해를 입으면 20이 된다")
    void damageSubtractsAmount() {
        Health health = new Health(30, 60);

        health.damage(10);

        assertEquals(20, health.getHp());
    }

    @Test
    @DisplayName("체력 5에서 10 피해를 입어도 0 아래로 내려가지 않는다")
    void damageStopsAtZero() {
        Health health = new Health(5, 60);

        health.damage(10);

        assertEquals(0, health.getHp());
    }
}
```

실습 풀이: 하한 0은 Math.max로

시작 상태의 실패 메시지:

```
체력 30에서 10 피해를 입으면 20이 된다 — UnsupportedOperationException: 실습: damage
체력 5에서 10 피해를 입어도 0 아래로 내려가지 않는다 — UnsupportedOperationException: 실습: damage
```

`Health.java`의 `damage()`를 이것으로 바꾼다.

```
public void damage(int amount) {
    currentHp = Math.max(currentHp - amount, 0);
}
```

`HealthDamageTest` 2건이 통과한다. `restore`의 상한은 `Math.min`, `damage`의 하한은 `Math.max`다. 체력의 범위 규칙이 모두 `Health` 안에 모였다.

정리: 역할은 인터페이스로, 구현 공유는 포함으로

문제	해법	아이템
신분이 둘인 SCV	역할을 인터페이스로 나눈다	20
다섯 곳에 복사된 체력 계산	`Health`로 떼어 포함하고 위임한다	18
부모 내부 구현에 묶인 `CountingUnitSet`	내부 `Set`에 저장을 맡기고 필요한 메서드만 제공한다	18

> 다음 문제: 지금 SCV는 `new Health(MAX_HP)`를 스스로 만들어서 언제나 체력 60에서 시작한다. 오늘 SCV 테스트도 상한만 확인할 수 있었다. 체력 1인 SCV를 10 수리하는 장면은 어떻게 시험할까?

3교시

빈사 상태의 SCV를 시험할 수 없다

체력 1인 SCV를 수리하는 장면을 테스트로 확인하고 싶다. 어디서 막힐까?

시험하고 싶은 것: 체력 1인 SCV를 10 수리하면 11

새 요구: 「빈사 상태의 SCV도 수리 한 번에 정확히 10만큼 회복한다」를 테스트로 남긴다. 발췌: 2교시 끝의 `SCV.java`(`heal`, `repairTarget` 생략).

```
public class SCV implements Healable, Repairable {
    private static final int MAX_HP = 60;

    private final Health health = new Health(MAX_HP);

    @Override
    public void repair(int amount) {
        health.restore(amount);
    }

    public int getHp() {
        return health.getHp();
    }
}
```

> 2교시 실습에서 `Health.damage`를 만들었지만, SCV에는 체력을 깎는 메서드가 없고 `health`를 밖으로 내주지도 않는다.

테스트를 써 보면: new SCV()는 언제나 60에서 시작한다

`ScvRepairTest.java` 새 테스트 파일을 만든다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class ScvRepairTest {

    @Test
    @DisplayName("체력 1인 SCV를 10 수리하면 11이 된다")
    void repairFromCriticalState() {
        SCV damaged = new SCV();   // 체력 1로 만들 방법이 없다

        damaged.repair(10);

        assertEquals(11, damaged.getHp());
    }
}
```

실행하면 `expected: <11> but was: <60>`으로 실패한다. SCV를 만드는 길은 `new SCV()` 하나뿐이고 그 안에서 체력 60이 정해진다. 출발 상태를 고를 수 없으니 이 테스트는 수리 규칙이 아니라 상한 규칙만 확인한다.

편법: setHp를 public으로 열면? 게임 규칙에 없는 문이 생긴다

테스트를 통과시키려고 `SCV.java`에 임시로 문을 연다. `health` 필드 줄을 아래 코드로 바꾼다(`final`도 뺐다).

```
    private Health health = new Health(MAX_HP);

    public void setHp(int hp) {   // 테스트용으로 열어 둔 문
        this.health = new Health(hp, MAX_HP);
    }
```

다음 장의 `ScvSetterTest` 2건은 **모두 통과한다**. 첫 번째는 체력 1에서 수리해 11, 두 번째는 `setHp(60)` 한 줄로 수리 없이 완전 회복이다. `ScvRepairTest`는 여전히 `new SCV()`라 빨간색 그대로다.

> 테스트 편의가 공개 API가 되어 「수리·치료로만 체력이 오른다」는 규칙이 깨졌다. 게임 코드 어디서든 `setHp(60)`을 부를 수 있다.

편법을 확인하는 테스트: ScvSetterTest 2건

ScvSetterTest.java

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class ScvSetterTest {

    @Test
    @DisplayName("setHp로 체력 1을 만들면 10 수리해 11이 된다")
    void repairAfterSetHp() {
        SCV damaged = new SCV();
        damaged.setHp(1);

        damaged.repair(10);

        assertEquals(11, damaged.getHp());
    }

    @Test
    @DisplayName("setHp가 public이면 수리 없이도 체력을 60으로 되돌릴 수 있다")
    void anyoneCanResetHp() {
        SCV scv = new SCV();
        scv.setHp(1);

        scv.setHp(60);   // 게임 코드 어디서든 호출할 수 있다

        assertEquals(60, scv.getHp());
    }
}
```

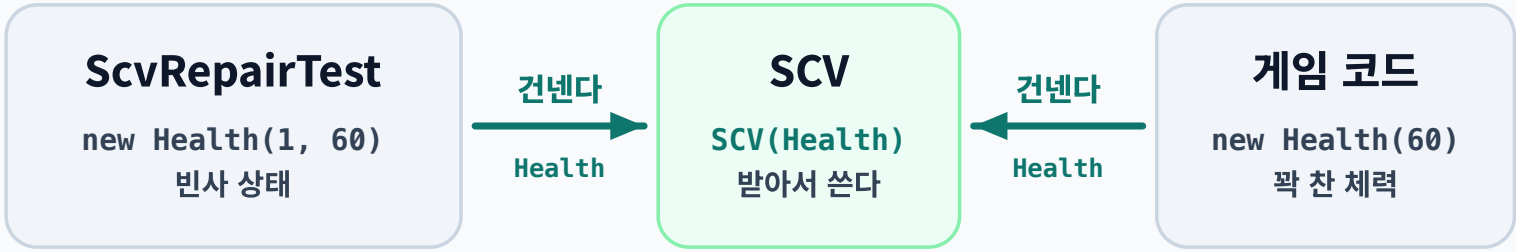
뿌리: SCV가 자기 Health를 직접 만든다



테스트는 `new SCV()` 만 부를 수 있다. 어떤 `Health` 를 쓸지는 SCV가 필드 초기화에서 `new Health(60)` 으로 혼자 정한다. 바깥에서 끼어들 틈이 없다.

> 현업 장면: `@Autowired` 필드 주입으로 협력 객체를 채우는 서비스를 스프링 없이 `new` 로 테스트하려다 막힌 적이 있다면, 지금 SCV가 같은 처지다.

만들지 말고 받는다



SCV는 `Health`를 만들지 않고 **생성자로 받는다**. 테스트는 체력 1짜리를, 게임 코드는 체력 60짜리를 만들어 건넨다. SCV 코드는 하나다. 필요한 협력 객체를 바깥에서 넣어 주는 것을 **의존 객체 주입**이라 한다.

> 이것이 이펙티브 자바 아이템 5: 자원을 직접 명시하지 말고 의존 객체 주입을 사용하라. 비유: 무기 공장이 탄창을 직접 찍지 않고 받아서 끼운다. 연습탄이든 실탄이든 같은 총이다. SOLID의 **DIP(의존 역전 원칙)**와는 다르다. DIP는 추상에 의존하라는 원칙이고, `Health`는 구체 클래스라 여기는 DI만 했다.

생성자 주입 최소 구현

SCV.java 앞의 `SCV.java`를 파일 전체로 다시 쓴다(편법 `setHp`도 사라진다). 필드 초기화가 사라지고 생성자 매개변수가 생겼다.

```
import java.util.Objects;

public class SCV implements Healable, Repairable {
    private static final int MAX_HP = 60;
    static final int REPAIR_AMOUNT = 10;

    private final Health health;

    public SCV(Health health) {
        this.health = Objects.requireNonNull(health, "Health는 필수입니다.");
    }
```

`Objects.requireNonNull`은 Java 7부터 있다. `null`을 받으면 그 자리에서 메시지와 함께 `NullPointerException`을 던진다. 받은 순간 막아야 나중에 `repair`에서 원인 모를 예외가 나지 않는다.

생성자 주입 최소 구현 (이어서)

SCV.java (이어서) 나머지 메서드는 2교시와 같다.

```
    @Override
    public void heal(int amount) {
        health.restore(amount);
    }

    @Override
    public void repair(int amount) {
        health.restore(amount);
    }

    public void repairTarget(Repairable target) {
        target.repair(REPAIR_AMOUNT);
    }

    public int getHp() {
        return health.getHp();
    }
}
```

기존 테스트를 새 생성자에 맞춘다

기본 생성자가 사라져서 `new SCV()`를 부르는 테스트는 모두 컴파일 에러가 난다. 아래처럼 고친다.

파일	지금	이렇게 바꾼다
`ScvRepairTest.java`	`new SCV()`	`new SCV(new Health(1, 60))`
`ScvStartTest.java`	`new SCV()`	`new SCV(new Health(60))`
`ScvRoleTest.java` (4곳)	`new SCV()`	`new SCV(new Health(60))`
`ScvSetterTest.java`	`setHp` 호출	파일을 지운다(`setHp`가 없다)
`VultureTest.java`, `MarineHealTest.java`	—	그대로(`SCV.REPAIR_AMOUNT`, `new Marine()`은 안 바뀌었다)

테스트 통과: new SCV(new Health(1, 60))을 10 수리하면 11

발췌: 고친 `ScvRepairTest.java`

```
@Test
@DisplayName("체력 1인 SCV를 10 수리하면 11이 된다")
void repairFromCriticalState() {
    SCV damaged = new SCV(new Health(1, 60));

    damaged.repair(10);

    assertEquals(11, damaged.getHp());
}
```

> 통과한다. 프로젝트 전체 13건이 초록이다. `setHp` 같은 문을 열지 않고도 원하는 출발 상태를 만들었다. 체력 범위 검사는 여전히 `Health` 생성자가 한다.

이 클래스들은 무엇을 직접 만들고 있나? (퀴즈)

퀴즈

세 클래스가 모두 필드 초기화에서 협력 객체를 직접 만든다. 테스트할 때 **바꾸고 싶어질 것**은 어느 것일까?

1. 치명타 판정기가 `new Random()`으로 주사위를 굴린다
2. 리플레이 기록기가 `new Date()`로 지금 시각을 찍는다
3. SCV가 `new Health(60)`으로 체력을 만든다

모두 고르시오.

정답: 셋 다 받는 것이 낫다

정답

셋 다 테스트에서 원하는 값으로 고정하고 싶은 대상이다.

직접 만드는 것	테스트에서 원하는 것	받는 방법(Java 8)
`new Random()`	매번 같은 주사위 결과	생성자로 `Random`을 받는다. 테스트는 `new Random(42)`
`new Date()`	고정된 「지금」	생성자로 `java.time.Clock`을 받는다. 테스트는 `Clock.fixed(...)`
`new Health(60)`	체력 1 같은 출발 상태	오늘 본 `SCV(Health health)`

> 기준: 실행할 때마다 달라지거나, 테스트에서 다른 상태로 시작하고 싶은 것은 받는다. 반대로 `new ArrayList<>()` 처럼 바꿀 이유가 없는 내부 도구까지 다 받을 필요는 없다.

새 불편: 모든 호출부가 new SCV(new Health(60))을 쓴다

발췌: 앞에서 고친 `ScvRoleTest.java`. 2교시까지 `new SCV()`였던 곳이 전부 이렇게 바뀌었다.

```
SCV scv = new SCV(new Health(60));
medic.treat(scv);

SCV repairer = new SCV(new Health(60));
SCV target = new SCV(new Health(60));
repairer.repairTarget(target);

SCV scv = new SCV(new Health(60));
scv.repairTarget(tank);
```

`60`은 SCV의 최대 체력인데 `MAX_HP`는 SCV 안의 `private` 상수라, 호출부마다 숫자로 외워 적는다. `new SCV(new Health(1, 60))`의 `1`과 `60`도 순서를 외워야 읽힌다.

> 더 나쁘게는 `new SCV(new Health(70))`도 예외 없이 만들어진다. 「최대 체력 60」 규칙을 이제 SCV가 아니라 호출부가 지킨다.

생성자에는 이름이 없다: 시그니처가 같은 두 생성자는 못 만든다

그럼 `new SCV(1)` 처럼 숫자 하나만 받는 생성자를 두면? 「다친 SCV」와 「최대 체력을 정한 SCV」를 둘 다 원하면 이렇게 된다.

```
public SCV(int currentHp) {                // 다친 SCV
    this.health = new Health(currentHp, MAX_HP);
}

public SCV(int maxHp) {                    // 최대 체력을 정한 SCV
    this.health = new Health(maxHp);
}
```

```
error: constructor SCV(int) is already defined in class SCV
```

생성자 이름은 클래스 이름으로 고정이라, 구분할 수단이 매개변수 타입뿐이다. 매개변수 이름은 시그니처에 들어가지 않는다.

이름 있는 생성 메서드



SCV를 만드는 입구에 **이름**을 붙인다. `create()`는 꽉 찬 체력으로, `ofDamaged(1)`은 다친 상태로 만든다. 두 입구 모두 `MAX_HP`를 SCV 안에서 채워 넣고, 실제 생성은 바깥에 숨긴 생성자 하나가 맡는다. 이런 `static` 생성 메서드를 **정적 팩토리 메서드**라 한다.

> 이것이 이펙티브 자바 아이템 1: 생성자 대신 정적 팩토리 메서드를 고려하라. 디자인 패턴의 팩토리 메서드 패턴과는 다른 것이다.

정적 팩토리 최소 구현: create, ofDamaged, package-private 생성자

SCV.java 앞의 `SCV.java`를 고친다. `public SCV(Health health)` 생성자를 아래 세 메서드로 바꾼다. 나머지 (import, 필드, `heal`, `repair`, `repairTarget`, `getHp`)는 그대로다.

```
    SCV(Health health) {
        this.health = Objects.requireNonNull(health, "Health는 필수입니다.");
    }

    public static SCV create() {
        return new SCV(new Health(MAX_HP));
    }

    public static SCV ofDamaged(int currentHp) {
        return new SCV(new Health(currentHp, MAX_HP));
    }
```

생성자에서 `public`을 뺐다(package-private). 같은 패키지에서는 생성자도 쓸 수 있지만, 다른 패키지에서는 이름 있는 두 정적 팩토리만 쓸 수 있다. `null` 검사는 생성자에 그대로 둔다.

테스트가 문장처럼 읽힌다

	주입만 했을 때	정적 팩토리
다친 SCV	`new SCV(new Health(1, 60))`	`SCV.ofDamaged(1)`
멀쩡한 SCV	`new SCV(new Health(60))`	`SCV.create()`
최대 체력 60을 아는 곳	호출부 전부	`SCV` 한 곳

`ScvRepairTest.java`(1곳)와 `ScvRoleTest.java`(4곳)의 생성 식을 표의 오른쪽으로 바꾼다. `ScvStartTest.java`는 지운다(같은 확인을 다음 장 `ScvFactoryTest`가 한다). 발췌: 고친 `ScvRepairTest.java`

```
@Test
@DisplayName("체력 1인 SCV를 10 수리하면 11이 된다")
void repairFromCriticalState() {
    SCV damaged = SCV.ofDamaged(1);

    damaged.repair(10);

    assertEquals(11, damaged.getHp());
}
```

ScvFactoryTest: 최대 체력 규칙은 SCV가 지킨다

`ScvFactoryTest.java` 새 테스트 파일을 만든다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class ScvFactoryTest {

    @Test
    @DisplayName("create로 만든 SCV는 체력 60에서 시작한다")
    void createStartsAtMaxHp() {
        SCV scv = SCV.create();

        assertEquals(60, scv.getHp());
    }

    @Test
    @DisplayName("ofDamaged(61)은 최대 체력을 넘으므로 만들 수 없다")
    void ofDamagedRejectsOverMaxHp() {
        assertThrows(IllegalArgumentException.class, () -> SCV.ofDamaged(61));
    }
}
```

> SCV 테스트 모두 통과(전체 14건). 다른 패키지에서는 체력 70인 SCV를 못 만든다.

이미 매일 쓰는 정적 팩토리와 이름 관례

이름	뜻	JDK 예 (Java 8)
`of`	여러 값을 모아 만든다	`LocalDate.of(2026, 10, 25)`, `EnumSet.of(a, b)`
`from`	다른 타입 하나를 변환	`Date.from(instant)`
`valueOf`	`from`·`of`의 긴 이름. 캐싱하기도	`Integer.valueOf(7)`, `Boolean.valueOf(true)`
`getInstance`	같은 객체일 수도 있다	`Calendar.getInstance()`
`newInstance` / `create`	매번 새 객체	`Array.newInstance(...)`, `SCV.create()`
`newType` / `type`	다른 클래스에 둘 때	`Files.newBufferedReader(path)`, `Collections.list(e)`

생성자 대신 얻는 것: **이름**, **캐싱**(`Boolean.valueOf`), **하위 타입 반환**(`Collections.unmodifiableList` 는 구현 클래스를 숨기고 `List` 로 돌려준다). 대신 `public` 생성자가 없으면 상속할 수 없다.

> 현업 장면: `LocalDate.of` , `Collections.emptyList` 처럼 이미 매일 이름 있는 생성을 쓰고 있었다.

Boolean.valueOf(true) == Boolean.valueOf(true)의 결과는? (퀴즈)

퀴즈

```
boolean a = Boolean.valueOf(true) == Boolean.valueOf(true);
boolean b = new Boolean(true) == new Boolean(true);
```

`==` 는 값이 아니라 **같은 객체인지**를 비교한다.

1. `a` 도 `b` 도 `true`
2. `a` 는 `true` , `b` 는 `false`
3. `a` 는 `false` , `b` 는 `true`
4. `a` 도 `b` 도 `false`

정답: 매번 새로 만들지 않아도 된다(캐싱)

정답

정답은 2번이다. 다음 장의 `ValueOfCacheTest`로 확인한다.

- `Boolean.valueOf(true)`는 미리 만들어 둔 `Boolean.TRUE`를 돌려준다. 몇 번을 불러도 같은 객체다.
- `new Boolean(true)`는 생성자라서 부를 때마다 새 객체를 만든다. 생성자는 「새 객체」 말고는 약속할 수 없다.

> 정적 팩토리는 새로 만들지, 있는 것을 돌려줄지를 클래스가 고를 수 있다. 그렇다고 래퍼 객체를 `==`로 비교하지는 않는다. `Integer.valueOf`는 -128~127만 캐싱해서 그 밖에서는 `==`가 `false`가 된다. 값 비교는 `equals`(7교시)다.

확인: ValueOfCacheTest

`ValueOfCacheTest.java` 새 테스트 파일을 만든다. 2건 모두 통과한다(전체 16건).

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertTrue;

class ValueOfCacheTest {

    @Test
    @DisplayName("Boolean.valueOf(true)는 매번 같은 객체를 돌려준다")
    void valueOfReturnsCachedInstance() {
        assertTrue(Boolean.valueOf(true) == Boolean.valueOf(true));
        assertTrue(Boolean.valueOf(true) == Boolean.TRUE);
    }

    @Test
    @DisplayName("new Boolean(true)는 부를 때마다 새 객체를 만든다")
    void constructorCreatesNewInstance() {
        assertFalse(new Boolean(true) == new Boolean(true));
    }
}
```

마린에게도 주입과 정적 팩토리를 (실습)

실습 파일 수정: 앞에서 만든 `Marine.java` · 테스트: `MarineTest.java` (다음 두 장)

먼저 `Marine.java`의 `heal` 위에 빈 껍데기 두 메서드를 더한다.

```
    public static Marine create() {
        throw new UnsupportedOperationException("실습: create");
    }

    public static Marine ofDamaged(int currentHp) {
        throw new UnsupportedOperationException("실습: ofDamaged");
    }
```

1. 필드 초기화를 지우고 package-private 생성자 `Marine(Health health)`를 만든다(`requireNonNull` 포함).
2. `create()`, `ofDamaged(int)`를 SCV처럼 채운다. `Marine`의 최대 체력은 40이다.
3. 기본 생성자가 사라지면 `MarineHealTest.java`의 `new Marine()`이 컴파일되지 않는다. `Marine.create()`로 바꾼다.

왜: 마린의 최대 체력 40도 마린 한 곳만 안다.

실습용 테스트: MarineTest

`MarineTest.java` 새 테스트 파일을 만든다.

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class MarineTest {

    @Test
    @DisplayName("체력 5인 마린을 메딕이 치료하면 15가 된다")
    void medicHealsDamagedMarine() {
        Marine marine = Marine.ofDamaged(5);
        Medic medic = new Medic();

        medic.treat(marine);

        assertEquals(15, marine.getHp());
    }
```

실습용 테스트: MarineTest (이어서)

`MarineTest.java` (이어서) 앞 장 코드 바로 아래에 붙인다.

```
    @Test
    @DisplayName("체력 41인 마린은 만들 수 없다")
    void rejectsHpOverMaxHp() {
        assertThrows(IllegalArgumentException.class, () -> Marine.ofDamaged(41));
    }

    @Test
    @DisplayName("create로 만든 마린은 체력 40에서 시작한다")
    void createStartsAtMaxHp() {
        Marine marine = Marine.create();

        assertEquals(40, marine.getHp());
    }
}
```

빈 껍데기 상태로 실행하면 3건 모두 빨간색이다(`UnsupportedOperationException: 실습: ofDamaged` · `실습: create`). `() -> Marine.ofDamaged(41)`은 실행할 코드를 넘기는 문법이다(Day 2에서 자세히).

실습 풀이: SCV와 같은 모양

Marine.java 파일 전체. 바꾼 뒤에는 `MarineTest` 3건이 초록이다.

```
import java.util.Objects;

public class Marine implements Healable {
    private static final int MAX_HP = 40;

    private final Health health;

    Marine(Health health) {
        this.health = Objects.requireNonNull(health, "Health는 필수입니다.");
    }

    public static Marine create() {
        return new Marine(new Health(MAX_HP));
    }

    public static Marine ofDamaged(int currentHp) {
        return new Marine(new Health(currentHp, MAX_HP));
    }

    @Override
    public void heal(int amount) {
        health.restore(amount);
    }

    public int getHp() {
        return health.getHp();
    }
}
```

3교시 끝: 프로젝트 상태

클래스	만드는 법
`SCV`	`SCV.create()`, `SCV.ofDamaged(hp)` (생성자 `SCV(Health)`는 package-private)
`Marine`	`Marine.create()`, `Marine.ofDamaged(hp)` (생성자 `Marine(Health)`는 package-private)
`SiegeTank`, `Vulture`, `Medic`	그대로 `new`
`Health`, `CountingUnitSet`	그대로 `new`

테스트 10개 파일: `ScvRepairTest`, `ScvRoleTest`, `ScvFactoryTest`, `ValueOfCacheTest`, `MarineTest`, `MarineHealTest`, `VultureTest`, `HealthTest`, `HealthDamageTest`, `CountingUnitSetTest`.

> 전체 19건 초록. 다음 문제: 같은 집결지에서 출발한 SCV 두 기는?

4교시

SCV 두 기가 같이 순간이동한다

한 기만 옮겼는데 다른 한 기도 따라 움직였다. 누가 옮긴 걸까?

새 요구: SCV가 위치를 가진다

같은 집결지 `(10, 20)`에서 SCV 여러 기가 출발한다. 좌표를 담을 클래스부터 만든다.

MutablePosition.java 새 파일을 만든다.

```
public class MutablePosition {
    private int x;
    private int y;

    public MutablePosition(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() {
        return x;
    }

    public int getY() {
        return y;
    }

    public void setX(int x) {
        this.x = x;
    }

    public void setY(int y) {
        this.y = y;
    }
}
```

SCV가 위치를 받는다: 생성자와 팩토리

SCV.java 앞의 `SCV.java`를 파일 전체로 다시 쓴다. 3교시의 `SCV(Health)`가 `SCV(Health, MutablePosition)`로 바뀌고, `create()`·`ofDamaged()`는 `(0, 0)`에서 출발한다. 집결지에서 출발하는 `at`이 새로 생겼다.

```java
import java.util.Objects;

public class SCV implements Healable, Repairable {
    private static final int MAX_HP = 60;
    static final int REPAIR_AMOUNT = 10;

    private final Health health;
    private final MutablePosition position;

    SCV(Health health, MutablePosition position) {
        this.health = Objects.requireNonNull(health, "Health는 필수입니다.");
        this.position = Objects.requireNonNull(position);
    }

    public static SCV create() {
        return new SCV(new Health(MAX_HP), new MutablePosition(0, 0));
    }

    public static SCV ofDamaged(int currentHp) {
        return new SCV(new Health(currentHp, MAX_HP), new MutablePosition(0, 0));
    }

    public static SCV at(MutablePosition position) {
        return new SCV(new Health(MAX_HP), position);
    }
```

SCV가 위치를 받는다 (이어서)

SCV.java (이어서) 끝에 `getPosition()`이 생겼다. 나머지는 3교시와 같다.

```
    @Override
    public void heal(int amount) {
        health.restore(amount);
    }

    @Override
    public void repair(int amount) {
        health.restore(amount);
    }

    public void repairTarget(Repairable target) {
        target.repair(REPAIR_AMOUNT);
    }

    public int getHp() {
        return health.getHp();
    }

    public MutablePosition getPosition() {
        return position;
    }
}
```

> 기존 테스트 19건은 고칠 곳이 없다. 모두 `SCV.create()` · `ofDamaged()`만 불러서, 생성자가 바뀐 것을 모른다. 3교시 정적 팩토리의 덕이다.

scv1만 옮겼는데 scv2도 x = 99

`ScvPositionTest.java` 새 테스트 파일을 만든다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class ScvPositionTest {

    @Test
    @DisplayName("집결지를 공유한 SCV 하나를 옮겨도 다른 SCV는 그대로다")
    void movingOneScvKeepsOtherAtRally() {
        MutablePosition rally = new MutablePosition(10, 20);
        SCV scv1 = SCV.at(rally);
        SCV scv2 = SCV.at(rally);

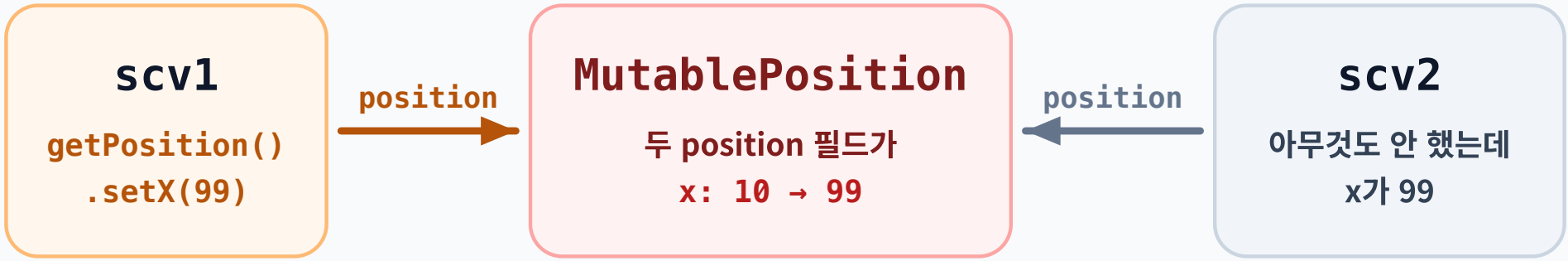
        scv1.getPosition().setX(99);

        assertEquals(10, scv2.getPosition().getX());
    }
}
```

```
expected: <10> but was: <99>
```

scv2에는 아무 명령도 내리지 않았다. 그런데 scv2의 x가 99다.

두 SCV가 위치 객체 하나를 함께 쥐고 있다



`SCV.at`은 받은 객체를 그대로 보관하고, `getPosition()`은 그 객체를 그대로 내준다.

> 현업 장면: getter로 내준 `List` · `Date`를 호출부가 고쳐 엔티티가 바뀐 버그와 같다.

들어올 때, 나갈 때 복사본을 만든다



이펙티브 자바 아이템 50: 적시에 방어적 복사본을 만들라. 앞의 `SCV.java`에서 생성자와 `getPosition()`을 아래로 바꾼다.

```
    SCV(Health health, MutablePosition position) {
        this.health = Objects.requireNonNull(health, "Health는 필수입니다.");
        this.position = new MutablePosition(position.getX(), position.getY());
    }

    public MutablePosition getPosition() {
        return new MutablePosition(position.getX(), position.getY());
    }
```

`ScvPositionTest`를 다시 실행하면 통과한다(전체 20건).

새 요구: 지정한 좌표로 이동하는 moveTo

복사 덕분에 `ScvPositionTest`는 통과한다. 이번에는 지정한 좌표로 이동하는 메서드를 더한다. `SCV.java`의 `position` 필드 줄에서 `final`을 빼고(`moveTo`가 다시 대입한다), 파일 끝 `getPosition()` 아래에 `moveTo`를 추가한다.

```
    private MutablePosition position;

    public void moveTo(MutablePosition target) {
        this.position = target;
    }
```

호출부가 이동 명령에 쓴 좌표 객체를, 다음 명령에 쓰려고 고쳐 쓰면 어떻게 될까?

moveTo를 하나 추가했더니 다시 뚫렸다

`ScvMoveTest.java` 새 테스트 파일을 만든다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class ScvMoveTest {

    @Test
    @DisplayName("이동 명령에 쓴 좌표 객체를 나중에 고쳐도 SCV는 그대로다")
    void reusingTargetDoesNotMoveScv() {
        MutablePosition target = new MutablePosition(30, 40);
        SCV scv = SCV.create();
        scv.moveTo(target);

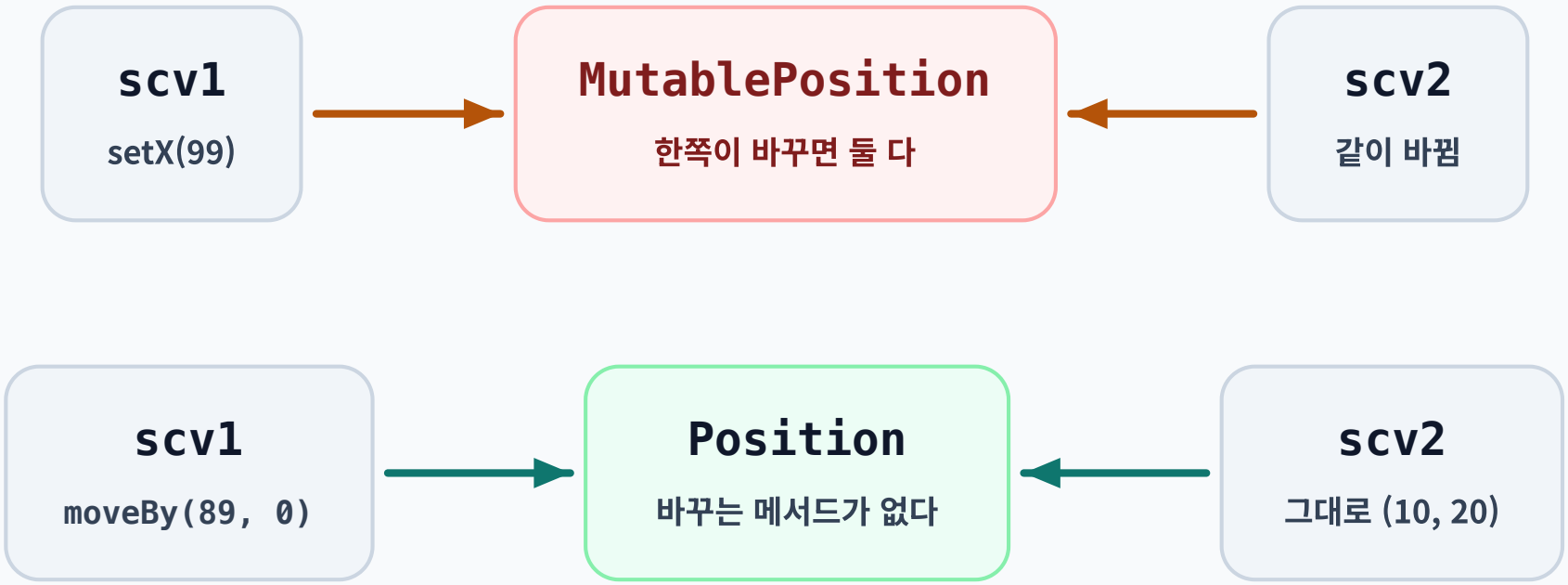
        target.setX(99);

        assertEquals(30, scv.getPosition().getX());
    }
}
```

```
expected: <30> but was: <99>
```

`moveTo`에서 복사를 빠뜨렸다. 가변 객체가 드나드는 메서드마다 복사해야 하고, 하나만 빠뜨려도 구멍이다. 복사할 필요 자체를 없앨 수는 없을까?

바뀌지 않는 위치라면 복사할 이유가 없다



바꾸는 메서드가 없으면 공유해도 안전하다. 옮길 때는 **새 좌표로 자기 필드만 갈아 끼운다**.

> 이런 객체를 불변 객체라 한다. 이펙티브 자바 아이템 17: 변경 가능성을 최소화하라.

불변 클래스의 규칙으로 Position을 만든다

`Position.java` 새 파일을 만든다.

```
public final class Position {                // 규칙 1: 상속으로 못 바꾸게 final
    private final int x;                     // 규칙 2: 모든 필드를 private final로
    private final int y;

    public Position(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }          // 규칙 3: setter를 두지 않는다
    public int getY() { return y; }

    public Position move(int dx, int dy) {   // 규칙 4: 바뀐 값은 새 객체로
        return new Position(x + dx, y + dy);
    }
}
```

규칙 5: 가변 필드는 복사해 저장하고, 접근자에서는 복사본을 반환한다(`Position`은 `int`뿐이라 해당 없음).

`rally.move(89, 0)`은 새 `(99, 20)`을 만들고 `rally`는 그대로다.

SCV는 새 Position으로 갈아 끼운다

SCV.java 앞의 `SCV.java`를 파일 전체로 다시 쓴다. `MutablePosition`이 모두 `Position`으로 바뀌고, 생성자의 복사 코드가 사라진다.

```java
import java.util.Objects;

public class SCV implements Healable, Repairable {
    private static final int MAX_HP = 60;
    static final int REPAIR_AMOUNT = 10;

    private final Health health;
    private Position position;

    SCV(Health health, Position position) {
        this.health = Objects.requireNonNull(health, "Health는 필수입니다.");
        this.position = Objects.requireNonNull(position);
    }

    public static SCV create() {
        return new SCV(new Health(MAX_HP), new Position(0, 0));
    }

    public static SCV ofDamaged(int currentHp) {
        return new SCV(new Health(currentHp, MAX_HP), new Position(0, 0));
    }

    public static SCV at(Position position) {
        return new SCV(new Health(MAX_HP), position);
    }
```

SCV는 새 Position으로 갈아 끼운다: moveTo, moveBy

SCV.java (이어서) 복사 코드가 없다. 바뀌는 것은 `position`이 가리키는 객체뿐이다.

```
    @Override
    public void heal(int amount) {
        health.restore(amount);
    }

    @Override
    public void repair(int amount) {
        health.restore(amount);
    }

    public void repairTarget(Repairable target) {
        target.repair(REPAIR_AMOUNT);
    }

    public int getHp() {
        return health.getHp();
    }

    public Position getPosition() {
        return position;
    }

    public void moveTo(Position target) {
        position = Objects.requireNonNull(target);
    }

    public void moveBy(int dx, int dy) {
        position = position.move(dx, dy);
    }
}
```

테스트도 Position으로: 복사 코드 없이 버그가 사라졌다

`ScvPositionTest.java`의 테스트 메서드를 이것으로 바꾼다. 좌표를 고치는 대신 `moveBy`로 옮긴다.

```
    @Test
    @DisplayName("집결지를 공유한 SCV 하나를 옮겨도 다른 SCV는 그대로다")
    void movingOneScvKeepsOtherAtRally() {
        Position rally = new Position(10, 20);
        SCV scv1 = SCV.at(rally);
        SCV scv2 = SCV.at(rally);

        scv1.moveBy(89, 0);

        assertEquals(99, scv1.getPosition().getX());
        assertEquals(10, scv2.getPosition().getX());
    }
```

`ScvMoveTest.java`의 테스트 메서드를 이것으로 바꾼다. 다음 좌표는 `move`로 새로 만든다.

```
    @Test
    @DisplayName("이동 명령에 쓴 좌표로 다음 좌표를 만들어도 SCV는 그대로다")
    void reusingTargetDoesNotMoveScv() {
        Position target = new Position(30, 40);
        SCV scv = SCV.create();
        scv.moveTo(target);

        Position next = target.move(69, 0);

        assertEquals(99, next.getX());
        assertEquals(30, scv.getPosition().getX());
    }
```

PositionTest: move는 새 좌표를 돌려준다

`PositionTest.java` 새 테스트 파일을 만든다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class PositionTest {

    @Test
    @DisplayName("move는 새 Position을 돌려주고 원래 좌표는 그대로 둔다")
    void moveReturnsNewPosition() {
        Position rally = new Position(10, 20);

        Position moved = rally.move(89, 0);

        assertEquals(99, moved.getX());
        assertEquals(10, rally.getX());
    }
}
```

> 프로젝트 전체 22건이 초록이다. `Position`에는 `setX`가 없어서 `getPosition().setX(99)` 처럼 고칠 방법 자체가 없다. `MutablePosition.java`는 이제 쓰는 곳이 없지만 지우지 않고 둔다. 7교시에 다시 꺼낸다.

다음 중 불변인 것을 모두 고르면? (퀴즈)

퀴즈

매일 쓰는 JDK 클래스다. 만든 뒤에 그 객체의 상태를 바꿀 방법이 있는지 떠올려 보자.

1. `String`
2. `java.time.LocalDate`
3. `java.util.Date`
4. `java.util.ArrayList`

정답: String과 LocalDate — Date는 가변이라 LocalDate가 나왔다

정답

정답은 1번과 2번이다.

- `String` : `toUpperCase()` , `replace()` 는 새 문자열을 돌려준다. 원래 문자열은 그대로다.
- `LocalDate` : `plusDays()` , `withYear()` 가 새 날짜를 돌려준다. Java 8에서 새로 들어왔다.
- `Date` : `setTime()` 으로 같은 객체의 시각이 바뀐다. 그래서 필드로 둘 때마다 방어적 복사가 필요했다.
- `ArrayList` : `add()` , `set()` 으로 내용이 바뀐다.

> 현업 장면: 캐시나 싱글턴 빈에 담아 둔 객체를 누군가 setter로 바꿔 다른 요청까지 오염된 적이 있다면, 그 객체가 불변이었으면 생기지 않았을 일이다.

새 요구: SCV 상태를 순간마다 기록한다 — ScvState

리플레이용으로 위치, 체력, 미네랄을 한 묶음으로 남긴다. `ScvState.java` 새 파일을 만든다.

```
public final class ScvState {
    private final Position position;
    private final Health health;
    private final int mineral;

    public ScvState(Position position, Health health, int mineral) {
        this.position = position;
        this.health = health;
        this.mineral = mineral;
    }

    public Position getPosition() { return position; }
    public Health getHealth() { return health; }
    public int getMineral() { return mineral; }
}
```

기록하려고 `SCV.java`의 `getHp()` 아래에 getter를 하나 추가한다.

```
    public Health getHealth() {
        return health;
    }
```

클래스는 `final`, 필드는 `private final`, setter도 없다. 불변 규칙을 다 지킨 것처럼 보인다.

기록해 둔 상태의 체력이 나중에 바뀌었다

`ScvStateTest.java` 새 테스트 파일을 만든다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class ScvStateTest {

    @Test
    @DisplayName("기록한 ScvState는 이후 수리에 영향받지 않는다")
    void recordedStateIgnoresLaterRepair() {
        SCV scv = SCV.ofDamaged(30);
        ScvState recorded = new ScvState(scv.getPosition(), scv.getHealth(), 50);

        scv.repair(10);

        assertEquals(30, recorded.getHealth().getHp());
    }
}
```

```
expected: <30> but was: <40>
```

`ScvState`와 SCV가 같은 가변 `Health`를 공유한다. SCV를 수리하면 기록의 체력도 함께 바뀐다.

2교시의 Health가 이번 문제 때문에 불변으로 바뀐다

Health.java 2교시에 만든 `Health.java`를 파일 전체로 다시 쓴다. 클래스에 `final`, 필드는 모두 `private final`이다. 두 생성자는 지금과 같다.

```
public final class Health {
    private final int currentHp;
    private final int maxHp;

    public Health(int maxHp) {
        this(maxHp, maxHp);
    }

    public Health(int currentHp, int maxHp) {
        if (maxHp <= 0 || currentHp < 0 || currentHp > maxHp) {
            throw new IllegalArgumentException("올바르지 않은 체력 값입니다.");
        }
        this.currentHp = currentHp;
        this.maxHp = maxHp;
    }
```

restore와 damage는 새 Health를 돌려준다

Health.java (이어서) 2교시에는 `currentHp = ...;`로 자기 값을 바꿨다. 이제는 바뀐 값으로 `new Health(...)`를 돌려주고 자기는 그대로다.

```
    public Health restore(int amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("회복량은 0 이상이어야 합니다.");
        }
        return new Health(Math.min(currentHp + amount, maxHp), maxHp);
    }

    public Health damage(int amount) {
        return new Health(Math.max(currentHp - amount, 0), maxHp);
    }

    public int getHp() {
        return currentHp;
    }

    public int getMaxHp() {
        return maxHp;
    }
}
```

받는 쪽은 돌려받은 Health를 다시 대입한다

`Health` 만 바꾸고 실행하면 컴파일은 되지만 6건이 빨간색이다. `health.restore(amount);` 가 새 `Health` 를 받아 놓고 버리기 때문이다. 네 파일을 표대로 고친다.

파일	지금	이렇게 바꾼다
`SCV.java` , `Marine.java` 필드	`private final Health health;`	`private Health health;`
`SiegeTank.java` , `Vulture.java` 필드	`private final Health health = new Health(MAX_HP);`	`private Health health = new Health(MAX_HP);`
`SCV` 의 `heal` · `repair` , `Marine` 의 `heal` , `SiegeTank` · `Vulture` 의 `repair`	`health.restore(amount);`	`health = health.restore(amount);`

이제 `ScvRepairTest` , `MarineTest` 가 다시 초록이다. 남은 4건은 `Health` 를 직접 고치던 `HealthTest` , `HealthDamageTest` 다. 다음 장으로 바꾼다.

HealthTest, HealthDamageTest를 불변에 맞춘다

두 테스트 파일을 아래로 통째로 바꾼다. 돌려받은 새 `Health`를 확인하고, 원래 객체는 그대로인지도 본다.

HealthTest.java

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class HealthTest {

    @Test
    @DisplayName("체력 30에서 10 회복하면 40인 새 Health가 나온다")
    void restoreReturnsNewHealth() {
        Health health = new Health(30, 60);

        Health restored = health.restore(10);

        assertEquals(40, restored.getHp());
        assertEquals(30, health.getHp());
    }

    @Test
    @DisplayName("체력 55에서 10 회복해도 최대 체력 60에서 멈춘다")
    void restoreStopsAtMaxHp() {
        Health health = new Health(55, 60);

        assertEquals(60, health.restore(10).getHp());
    }
}
```

HealthDamageTest.java

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class HealthDamageTest {

    @Test
    @DisplayName("체력 30에서 10 피해를 입으면 20인 새 Health가 나온다")
    void damageReturnsNewHealth() {
        Health health = new Health(30, 60);

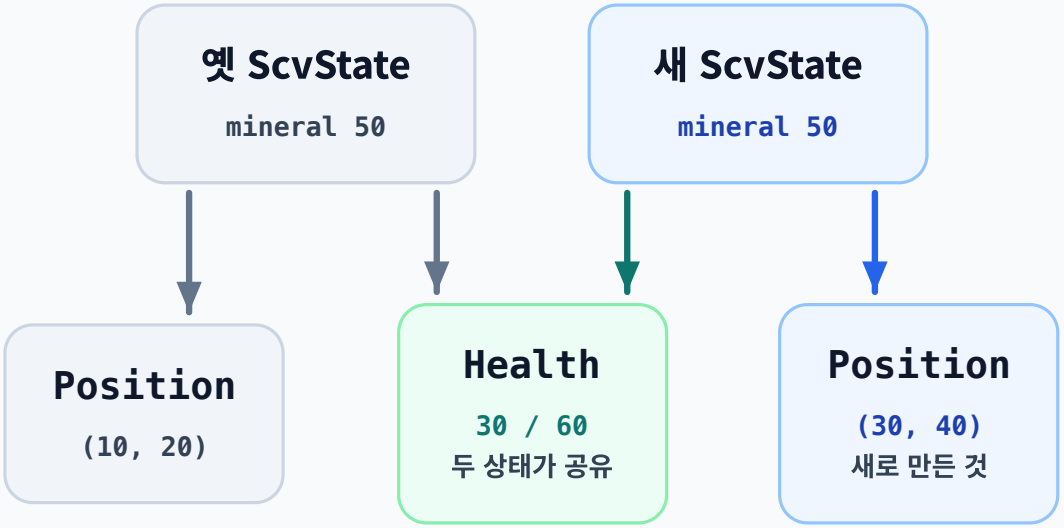
        Health damaged = health.damage(10);

        assertEquals(20, damaged.getHp());
        assertEquals(30, health.getHp());
    }

    @Test
    @DisplayName("체력 5에서 10 피해를 입어도 0 아래로 내려가지 않는다")
    void damageStopsAtZero() {
        Health health = new Health(5, 60);

        assertEquals(0, health.damage(10).getHp());
    }
}
```

하나만 바꾸고 나머지는 그대로 쓴다: 구조적 공유



프로젝트 전체 23건이 초록이다. 앞의 `ScvStateTest`는 고치지 않았는데 통과한다(기록은 30, SCV는 새 `Health`로 40). 새 기록에서는 위치를 새 객체로 만들고 불변 `Health`는 이전 기록과 공유한다. 바뀌지 않은 객체를 재사용하는 것을 구조적 공유라 한다.

withPosition, withMineral 구현 — LocalDate.withYear도 같은 방식

앞의 `ScvState.java` 끝(마지막 `}` 앞)에 두 메서드를 추가한다.

```
    public ScvState withPosition(Position newPosition) {
        return new ScvState(newPosition, this.health, this.mineral);
    }

    public ScvState withMineral(int newMineral) {
        return new ScvState(this.position, this.health, newMineral);
    }
```

JDK도 같은 방식이다. `WithYearDemo.java` 를 만들어 실행해 본다.

```
import java.time.LocalDate;

/** JDK의 불변 클래스도 withXxx로 하나만 바꾼 새 객체를 돌려준다. */
class WithYearDemo {
    public static void main(String[] args) {
        LocalDate today = LocalDate.of(2026, 10, 12);
        LocalDate nextYear = today.withYear(2027);

        System.out.println(today);
        System.out.println(nextYear);
    }
}
```

`2026-10-12` , `2027-10-12` 두 줄이 찍힌다. 연도만 바꾼 새 날짜가 나오고 원래 날짜는 그대로다.

ScvStateWithTest: 바꾸지 않은 필드는 같은 객체다

ScvStateWithTest.java 새 테스트 파일을 만든다. 2건 통과(전체 25건). `Health`가 불변이 됐기 때문에 공유해도 안전하다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertSame;

class ScvStateWithTest {

    @Test
    @DisplayName("withPosition은 위치만 새것이고 Health는 같은 객체를 공유한다")
    void withPositionSharesHealth() {
        ScvState before = new ScvState(new Position(10, 20), new Health(30, 60), 50);

        ScvState after = before.withPosition(new Position(30, 40));

        assertEquals(30, after.getPosition().getX());
        assertSame(before.getHealth(), after.getHealth());
    }

    @Test
    @DisplayName("withMineral은 미네랄만 바꾸고 원래 상태는 그대로 둔다")
    void withMineralKeepsOriginal() {
        ScvState before = new ScvState(new Position(10, 20), new Health(30, 60), 50);

        ScvState after = before.withMineral(58);

        assertEquals(58, after.getMineral());
        assertEquals(50, before.getMineral());
        assertSame(before.getPosition(), after.getPosition());
    }
}
```

withHealth를 추가하고 공유를 assertSame으로 확인한다 (실습)

실습 파일 수정: 앞에서 만든 `ScvState.java` · 테스트: `ScvStateWithHealthTest.java` (다음 장)

시작 코드: `withMineral` 아래에 붙인다.

```
    public ScvState withHealth(Health newHealth) {
        throw new UnsupportedOperationException("실습: withHealth");
    }
```

1. 다음 장의 `ScvStateWithHealthTest.java`를 만들어 실행한다. 지금은 2건 모두 빨간색이다.
2. 체력만 `newHealth`로 바꾼 새 `ScvState`를 돌려주게 고친다. 위치와 미네랄은 지금 것을 그대로 넘긴다.
3. 다시 실행해 「withHealth는 체력만 바뀐 새 상태를 돌려준다」와 「withHealth 뒤에도 Position은 같은 객체를 공유한다」 2건이 초록인지 확인한다.

왜: 수리 직후 상태를 기록할 때 위치 객체를 새로 만들 필요가 없다는 것을 `assertSame`이 보여 준다.

실습용 테스트: ScvStateWithHealthTest

`ScvStateWithHealthTest.java` 새 테스트 파일을 만든다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertSame;

class ScvStateWithHealthTest {

    @Test
    @DisplayName("withHealth는 체력만 바뀐 새 상태를 돌려준다")
    void withHealthChangesOnlyHealth() {
        ScvState before = new ScvState(new Position(10, 20), new Health(30, 60), 50);

        ScvState after = before.withHealth(before.getHealth().restore(10));

        assertEquals(40, after.getHealth().getHp());
        assertEquals(30, before.getHealth().getHp());
        assertEquals(50, after.getMineral());
    }

    @Test
    @DisplayName("withHealth 뒤에도 Position은 같은 객체를 공유한다")
    void withHealthSharesPosition() {
        ScvState before = new ScvState(new Position(10, 20), new Health(30, 60), 50);

        ScvState after = before.withHealth(new Health(60, 60));

        assertSame(before.getPosition(), after.getPosition());
    }
}
```

실습 풀이: 바뀐 하나만 새로 넘긴다

시작 상태의 실패 메시지:

```
withHealth는 체력만 바뀐 새 상태를 돌려준다 — UnsupportedOperationException: 실습: withHealth
withHealth 뒤에도 Position은 같은 객체를 공유한다 — UnsupportedOperationException: 실습: withHealth
```

`ScvState.java`의 `withHealth`를 이것으로 바꾼다.

```
    public ScvState withHealth(Health newHealth) {
        return new ScvState(this.position, newHealth, this.mineral);
    }
```

2건이 통과한다(전체 27건). `assertSame`은 값이 같은지가 아니라 **같은 객체인지**를 본다.

오전 체크포인트: DomainFailureTest 4건을 모두 초록으로

실습 오전에 만든 코드에 `DomainFailureTest.java`를 새로 작성해 실행한다(다음 두 장). 오전을 시작할 때의 레거시 코드였다면 4건 모두 빨간색이었다.

테스트 (`@DisplayName`)	레거시의 원인	해결한 교시
메딕이 마린을 치료해도 최대 체력 40을 넘지 않는다	상한 검사 누락	1·2교시 역할, `Health`
체력 1인 SCV를 10 수리하면 11이 된다	체력을 정할 방법 없음	3교시 주입, `ofDamaged`
집결지를 공유한 SCV 하나를 옮겨도 다른 SCV는 그대로다	가변 좌표 공유	4교시 불변 `Position`
기록한 ScvState는 이후 수리에 영향받지 않는다	가변 체력 공유	4교시 불변 `Health`

> 4건이 모두 통과하면 오전 체크포인트 완료다(프로젝트 전체 31건). 오전에 만든 역할·포함·주입·불변이 테스트 4건으로 확인됐다.

DomainFailureTest ①: 상한과 체력 주입

DomainFailureTest.java 새 테스트 파일을 만든다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class DomainFailureTest {

    @Test
    @DisplayName("메딕이 마린을 치료해도 최대 체력 40을 넘지 않는다")
    void medicHealsMarineUpToMaxHp() {
        Marine marine = Marine.create();
        Medic medic = new Medic();

        medic.treat(marine);

        assertEquals(40, marine.getHp());
    }

    @Test
    @DisplayName("체력 1인 SCV를 10 수리하면 11이 된다")
    void repairFromCriticalState() {
        SCV damaged = SCV.ofDamaged(1);

        damaged.repair(10);

        assertEquals(11, damaged.getHp());
    }
```

DomainFailureTest ②: 불변 좌표와 불변 체력

DomainFailureTest.java (이어서) 앞 장 코드 바로 아래에 붙인다.

```
    @Test
    @DisplayName("집결지를 공유한 SCV 하나를 옮겨도 다른 SCV는 그대로다")
    void movingOneScvKeepsOtherAtRally() {
        Position rally = new Position(10, 20);
        SCV scv1 = SCV.at(rally);
        SCV scv2 = SCV.at(rally);

        scv1.moveBy(89, 0);

        assertEquals(10, scv2.getPosition().getX());
    }

    @Test
    @DisplayName("기록한 ScvState는 이후 수리에 영향받지 않는다")
    void recordedStateIgnoresLaterRepair() {
        SCV scv = SCV.ofDamaged(30);
        ScvState recorded = new ScvState(scv.getPosition(), scv.getHealth(), 50);

        scv.repair(10);

        assertEquals(30, recorded.getHealth().getHp());
    }
}
```

5교시

숫자 일곱 개의 의미를 맞혀 보세요

`new UnitStats(60, 0, 7, 2.8, 1, 50, 0)` — 이 줄을 읽을 수 있나요?

유닛 스탯: 필수 하나, 선택 여섯

SCV, 마린, 시즈 탱크… 유닛마다 스탯을 정해야 한다.

- 정할 값은 **일곱 개**다.
- 반드시 있어야 하는 값은 **체력** 하나뿐이다.
- 나머지 여섯은 정하지 않으면 기본값을 쓴다.

이 값을 담는 불변 클래스 `UnitStats`를 만든다. 가장 익숙한 방법, 인자 일곱 개짜리 생성자부터.

수치는 이 강의용 설정이다.

new UnitStats(60, 0, 7, 2.8, 1, 50, 0) — 7은 시야인가 사거리인가

코드 리뷰 화면이라고 생각하고 읽어 보자(발췌). IDE 힌트는 없다.

```
UnitStats planned = new UnitStats(60, 0, 7, 2.8, 1, 50, 0);
//                                ?   ?  ?   ?   ?  ?   ?
```

- `0` 이 두 번, 무엇의 0인가? `1` 은 사거리인가 방어력인가?
- 답을 알려면 생성자 선언을 열어 인자 순서를 세어야 한다:
  `(hp, armor, sight, speed, range, mineralCost, gasCost)`
- 3교시에서 "생성자에는 이름이 없다"고 했다. 인자가 일곱이면 **숫자에도 이름이 없다.**

생성자를 늘려 보면: 점층적 생성자

"필요한 만큼만 넘기게 하자"며 인자 수가 하나씩 늘어나는 생성자를 계단처럼 쌓는다.

UnitStats.java

```
public final class UnitStats {
    private final int hp;
    private final int armor;
    private final int sight;
    private final double speed;
    private final int range;
    private final int mineralCost;
    private final int gasCost;

    public UnitStats(int hp) { this(hp, 0); }
    public UnitStats(int hp, int armor) { this(hp, armor, 5); }
    public UnitStats(int hp, int armor, int sight) { this(hp, armor, sight, 1.0); }
    public UnitStats(int hp, int armor, int sight, double speed) {
        this(hp, armor, sight, speed, 1);
    }
    public UnitStats(int hp, int armor, int sight, double speed, int range) {
        this(hp, armor, sight, speed, range, 0);
    }
    public UnitStats(int hp, int armor, int sight, double speed, int range, int mineralCost) {
        this(hp, armor, sight, speed, range, mineralCost, 0);
    }
```

점층적 생성자 (이어서): 마지막 생성자가 모든 필드를 채운다

UnitStats.java (이어서)

```
    public UnitStats(int hp, int armor, int sight, double speed, int range,
                     int mineralCost, int gasCost) {
        this.hp = hp;
        this.armor = armor;
        this.sight = sight;
        this.speed = speed;
        this.range = range;
        this.mineralCost = mineralCost;
        this.gasCost = gasCost;
    }

    public int getHp() { return hp; }
    public int getArmor() { return armor; }
    public int getSight() { return sight; }
    public double getSpeed() { return speed; }
    public int getRange() { return range; }
    public int getMineralCost() { return mineralCost; }
    public int getGasCost() { return gasCost; }
}
```

- 필드가 모두 `final`이라 **불변은 지킨다.** 하지만 가스 비용만 정하려 해도 기본값 다섯 개를 다시 적는다.
- 호출부는 여전히 숫자 나열이다. 순서를 바꿔 넣으면? 다음 장에서 확인한다.

시야와 사거리를 바꿔 넣어도 컴파일된다

기획서 값을 옮겨 적다가 `7`과 `1`의 자리를 바꿨다. 둘 다 `int`라 컴파일러는 조용하다. UnitStatsConstructorTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class UnitStatsConstructorTest {

    @Test
    @DisplayName("기획서대로 옮겨 적은 SCV 스탯은 시야 7, 사거리 1이다")
    void scvSightAndRange() {
        UnitStats planned = new UnitStats(60, 0, 7, 2.8, 1, 50, 0);
        UnitStats scv = new UnitStats(60, 0, 1, 2.8, 7, 50, 0);

        assertEquals(planned.getSight(), scv.getSight());
        assertEquals(planned.getRange(), scv.getRange());
    }
}
```

실행 결과: `FAILED  expected: <7> but was: <1>`

> 테스트가 있어서 잡혔다. 없었다면 시야 1, 사거리 7인 SCV가 그대로 나간다.

생성자 계단에 흩어진 기본값을 모아 보면

생성자 계단의 `this(...)` 호출마다 숨어 있던 기본값을 한 표로 모았다. 이 표를 코드 한 곳에 두고 싶다.

필드	뜻	필수	기본값	SCV 값
`hp`	체력	필수	—	60
`armor`	방어력	선택	0	0
`sight`	시야	선택	5	7
`speed`	이동 속도	선택	1.0	2.8
`range`	사거리	선택	1	1
`mineralCost`	미네랄 비용	선택	0	50
`gasCost`	가스 비용	선택	0	0

인자 순서는 표의 위에서 아래 순서와 같다. 이 순서를 외워야만 호출부를 읽을 수 있다.

setter로 채우면: 자바빈즈

기본 생성자로 빈 객체를 만들고, setter로 값을 하나씩 채우는 방식이다. 필드에 `final`이 없다.

UnitStatsBean.java

```
public class UnitStatsBean {
    private int hp, armor, sight, range, mineralCost, gasCost;
    private double speed;

    public UnitStatsBean() { }

    public void setHp(int hp) { this.hp = hp; }
    public void setArmor(int armor) { this.armor = armor; }
    public void setSight(int sight) { this.sight = sight; }
    public void setSpeed(double speed) { this.speed = speed; }
    public void setRange(int range) { this.range = range; }
    public void setMineralCost(int mineralCost) { this.mineralCost = mineralCost; }
    public void setGasCost(int gasCost) { this.gasCost = gasCost; }

    public int getHp() { return hp; }
    public int getArmor() { return armor; }
    public int getSight() { return sight; }
    public double getSpeed() { return speed; }
    public int getRange() { return range; }
    public int getMineralCost() { return mineralCost; }
    public int getGasCost() { return gasCost; }
}
```

자바빈즈: 값마다 이름이 붙는다

UnitStatsBeanTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class UnitStatsBeanTest {

    @Test
    @DisplayName("setter로 이름을 붙여 채운 SCV는 시야 7이다")
    void namedSetters() {
        UnitStatsBean scv = new UnitStatsBean();
        scv.setHp(60);
        scv.setSight(7);
        scv.setSpeed(2.8);
        scv.setRange(1);
        scv.setMineralCost(50);

        assertEquals(7, scv.getSight());
    }
}
```

- 통과. 값마다 이름이 붙어 시야와 사거리를 헷갈릴 일이 없다. `armor`, `gasCost`처럼 기본값이면 부르지 않는다.

그런데 setHp를 빠뜨려도 객체는 만들어진다

`UnitStatsBeanTest.java` 에 이어서 쓴다(클래스 안, 앞 테스트 아래).

```
    @Test
    @DisplayName("setHp를 빠뜨려도 SCV는 만들어지고, 체력은 60이어야 한다")
    void forgottenRequiredSetter() {
        UnitStatsBean scv = new UnitStatsBean();
        scv.setSight(7);
        scv.setSpeed(2.8);
        scv.setRange(1);
        scv.setMineralCost(50);

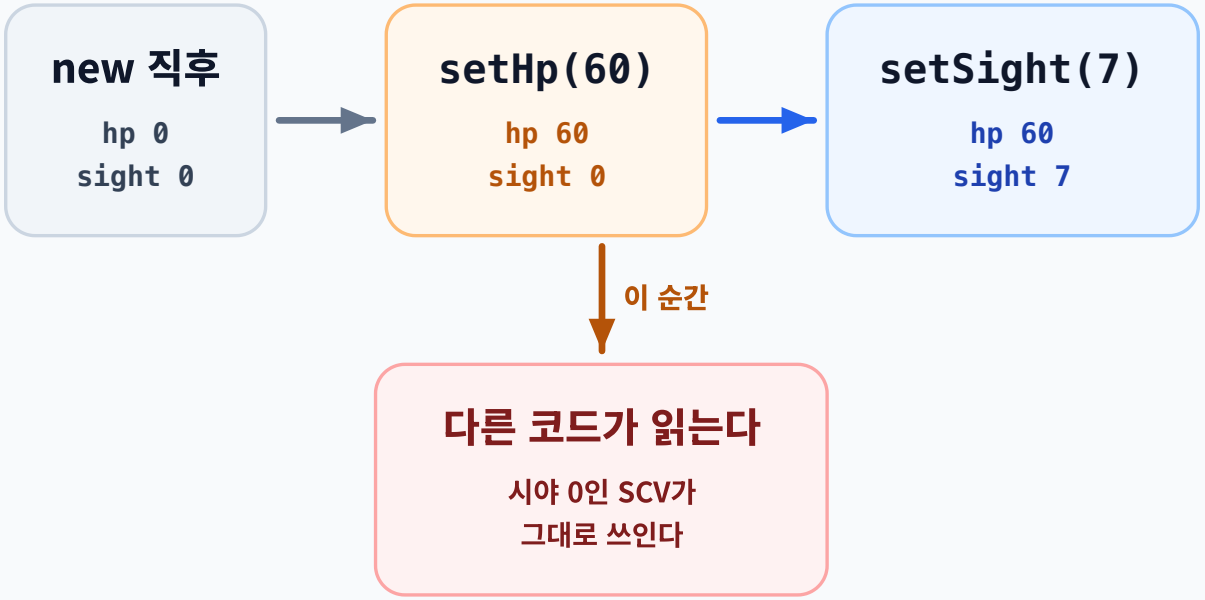
        assertEquals(60, scv.getHp());
    }
```

```
FAILED  expected: <60> but was: <0>
```

체력이 필수인데 컴파일러도 생성자도 막지 못한다. 0은 `int` 필드의 초기값이다.

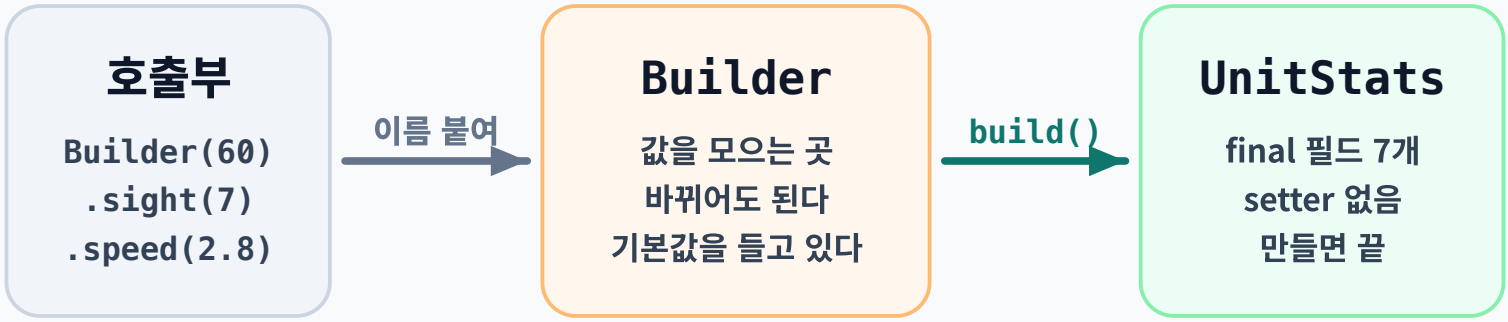
> 현업에서 본 장면: setter로 하나씩 채우는 VO/DTO. 생성자 인자 순서를 바꿔 넣어 데이터가 어긋난 뒤로 이 방식으로 바꾼 팀이 많다.

자바빈즈의 대가: 반쯤 만들어진 객체, 그리고 불변 포기



- `setHp`를 빠뜨려도 객체는 만들어진다: `expected: <60> but was: <0>`
- setter가 있으니 `final`을 붙일 수 없다. 4교시의 불변 규칙을 포기한다.

이름 붙여 하나씩 모은 뒤 한 번에 만든다



값은 `Builder`가 이름 붙여 모으고, `build()`가 불변 `UnitStats`를 한 번에 만든다. 이것이 **빌더 패턴**이다.

> 비유: 주문서에 항목마다 체크하고, 다 적은 뒤 한 번에 주문한다.

> 이것이 이펙티브 자바 아이템 2: 생성자에 매개변수가 많다면 빌더를 고려하라.

빌더 구현 1: UnitStats — private 생성자와 final 필드

먼저 지운다: `UnitStatsConstructorTest.java`, `UnitStatsBean.java`, `UnitStatsBeanTest.java`. 사라질 생성자를 부르거나 일부러 실패하는 시연용이라, 남기면 컴파일이 깨진다.

UnitStats.java 앞의 파일을 빌더 버전으로 **처음부터 다시 쓴다.**

```
public final class UnitStats {
    private final int hp;
    private final int armor;
    private final int sight;
    private final double speed;
    private final int range;
    private final int mineralCost;
    private final int gasCost;

    private UnitStats(Builder builder) {
        this.hp = builder.hp;
        this.armor = builder.armor;
        this.sight = builder.sight;
        this.speed = builder.speed;
        this.range = builder.range;
        this.mineralCost = builder.mineralCost;
        this.gasCost = builder.gasCost;
    }
```

빌더 구현 2: Builder — 필수는 생성자로, 선택은 메서드로, return this

UnitStats.java (이어서) `UnitStats` 안의 `static` 클래스라 `private` 생성자를 부를 수 있다. `return this`라서 점(`.`)으로 이어 부른다.

```
    public static class Builder {
        private final int hp;           // 필수
        private int armor = 0;          // 선택: 기본값
        private int sight = 5;
        private double speed = 1.0;
        private int range = 1;
        private int mineralCost = 0;
        private int gasCost = 0;

        public Builder(int hp) {
            if (hp <= 0) {
                throw new IllegalArgumentException("체력은 양수여야 합니다.");
            }
            this.hp = hp;
        }

        public Builder armor(int armor) { this.armor = armor; return this; }
        public Builder sight(int sight) { this.sight = sight; return this; }
        public Builder speed(double speed) { this.speed = speed; return this; }
        public Builder range(int range) { this.range = range; return this; }
        public Builder mineralCost(int mineralCost) { this.mineralCost = mineralCost; return this; }
        public Builder gasCost(int gasCost) { this.gasCost = gasCost; return this; }
```

빌더 구현 3: build()와 getter — setter는 없다

`UnitStats.java` (이어서)

```
        public UnitStats build() {
            // 검사는 잠시 뒤에
            return new UnitStats(this);
        }
    }

    public int getHp() { return hp; }
    public int getArmor() { return armor; }
    public int getSight() { return sight; }
    public double getSpeed() { return speed; }
    public int getRange() { return range; }
    public int getMineralCost() { return mineralCost; }
    public int getGasCost() { return gasCost; }
}
```

- 생성자가 `private`이라 밖에서는 `new UnitStats(...)`를 못 부른다. 만드는 길은 `Builder` 하나뿐이다.
- 기본값은 `Builder` 필드 초기값 한 곳에만 있다.
- 검사는 잠시 뒤 `build()`에 넣는다.

호출부가 읽힌다

전 `new UnitStats(60, 0, 7, 2.8, 1, 50, 0)` → **후** 빌더. 기본값인 `armor`, `range`, `gasCost`는 적지 않는다.

UnitStatsTest.java 빌더로 만든 SCV를 확인하는 첫 테스트다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class UnitStatsTest {

    @Test
    @DisplayName("이름 붙여 정한 값만 바뀌고 나머지는 기본값이다")
    void namedValuesOverrideDefaults() {
        UnitStats scv = new UnitStats.Builder(60)
                .sight(7)
                .speed(2.8)
                .mineralCost(50)
                .build();

        assertEquals(7, scv.getSight());
        assertEquals(1, scv.getRange());
        assertEquals(50, scv.getMineralCost());
    }
}
```

테스트로 확인: 지정하지 않은 값은 기본값

`UnitStatsTest.java` 에 이어서 쓴다(클래스 안, 앞 테스트 아래).

```
    @Test
    @DisplayName("체력만 정하면 나머지는 기본값이다")
    void defaultsWhenOnlyHpIsGiven() {
        UnitStats stats = new UnitStats.Builder(60).build();

        assertEquals(0, stats.getArmor());
        assertEquals(5, stats.getSight());
        assertEquals(1.0, stats.getSpeed());
        assertEquals(1, stats.getRange());
        assertEquals(0, stats.getMineralCost());
        assertEquals(0, stats.getGasCost());
    }
```

> 통과. 기본값은 `Builder` 필드 초기값 한 곳에만 있다. 점층적 생성자처럼 생성자 계단에 흩어지지 않는다.

불변식은 build()에서 한 번에 검사한다: 사거리 ≤ 시야

이 강의의 게임 규칙: **사거리는 시야보다 길 수 없다.** 어디서 검사할까?

- `range()` 안에서? `.range(10).sight(12)`는 `range(10)` 순간 시야가 아직 5라서 거절된다.
- 그래서 값을 **다 모은 뒤**, 객체를 만들기 직전에 한 번 검사한다.

`UnitStats.java`의 `build()`를 이것으로 바꾼다(`Builder` 안).

```
        public UnitStats build() {
            if (range > sight) {
                throw new IllegalArgumentException("사거리는 시야보다 길 수 없습니다.");
            }
            return new UnitStats(this);
        }
```

규칙을 어긴 `UnitStats`는 아예 만들어지지 않는다.

테스트: 사거리 10, 시야 7이면 IllegalArgumentException

`UnitStatsTest.java` 에 이어서 쓴다. `() -> ...` 는 3교시에서 본 "실행할 코드를 넘기는 문법"(Day 2에서 자세히).

```
    @Test
    @DisplayName("사거리 10, 시야 7이면 IllegalArgumentException이 난다")
    void rangeLongerThanSightIsRejected() {
        UnitStats.Builder builder = new UnitStats.Builder(150).sight(7).range(10);

        IllegalArgumentException e =
                assertThrows(IllegalArgumentException.class, () -> builder.build());

        assertEquals("사거리는 시야보다 길 수 없습니다.", e.getMessage());
    }

    @Test
    @DisplayName("호출 순서와 상관없이 다 모은 뒤에 검사한다")
    void checkAfterAllValuesAreCollected() {
        UnitStats tank = new UnitStats.Builder(150)
                .range(10)
                .sight(12)
                .build();

        assertEquals(10, tank.getRange());
    }
```

실습: 이동 속도는 0보다 커야 한다

실습 UnitStatsTest.java 에 이어서 두 테스트를 쓰고 실행한다. 체력 0 테스트는 바로 통과하고(`Builder` 생성자가 이미 막는다), 이동 속도 0 테스트는 실패한다.

```
    @Test
    @DisplayName("체력이 0이면 Builder를 만들 때 IllegalArgumentException이 난다")
    void zeroHpIsRejected() {
        IllegalArgumentException e =
                assertThrows(IllegalArgumentException.class, () -> new UnitStats.Builder(0));

        assertEquals("체력은 양수여야 합니다.", e.getMessage());
    }

    @Test
    @DisplayName("이동 속도가 0이면 IllegalArgumentException이 난다")
    void zeroSpeedIsRejected() {
        UnitStats.Builder builder = new UnitStats.Builder(60).speed(0);

        IllegalArgumentException e =
                assertThrows(IllegalArgumentException.class, () -> builder.build());

        assertEquals("이동 속도는 0보다 커야 합니다.", e.getMessage());
    }
```

실습 (이어서): build()에 이동 속도 검사를 넣는다

실습 시작 상태: 「이동 속도가 0이면 IllegalArgumentException이 난다」만 실패한다.

```
FAILED  Expected java.lang.IllegalArgumentException to be thrown, but nothing was thrown.
```

`UnitStats.java`의 `build()`를 이것으로 바꾼다. 사거리 검사 바로 아래에 속도 검사를 더했다.

```
        public UnitStats build() {
            if (range > sight) {
                throw new IllegalArgumentException("사거리는 시야보다 길 수 없습니다.");
            }
            if (speed <= 0) {
                throw new IllegalArgumentException("이동 속도는 0보다 커야 합니다.");
            }
            return new UnitStats(this);
        }
```

실행: `UnitStatsTest` 전체 실행 → 6건 모두 통과해야 한다.

> 규칙이 늘어도 검사 위치는 `build()` 한 곳이다. 호출부는 한 줄도 바뀌지 않는다.

그럼 우리 DTO에도 빌더를 다 달아야 하나요? (퀴즈)

퀴즈

현업 코드에는 필드 두세 개짜리 VO부터 필드 스무 개짜리 요청 DTO까지 있다. 빌더를 다는 기준으로 가장 알맞은 것은?

1. 모든 클래스에 단다. 일관성이 가장 중요하다.
2. 매개변수가 많고(대략 4개 이상) 선택 값이 섞여 있거나, 불변이 필요할 때 단다.
3. 빌더는 객체를 하나 더 만들어 느리므로 쓰지 않는다.
4. 필수 값이 하나도 없을 때만 단다.

풀이: 빌더를 쓸 때와 쓰지 않을 때

정답

정답은 2번이다.

상황	고를 것
인자 2~3개, 모두 필수	생성자 또는 정적 팩토리
생성 방법마다 뜻이 다르다 (`create` , `ofDamaged`)	정적 팩토리
인자 4개 이상, 선택 값이 많다, 같은 타입이 여럿	빌더
앞으로 필드가 늘어날 것 같다	처음부터 빌더
도구가 기본 생성자 + setter를 요구하는 DTO, 불변이 필요 없다	setter 유지도 괜찮다

> 빌더도 비용이 있다: 코드가 늘고, 필드를 추가할 때 두 곳을 고친다.

정리: 생성 방법 세 가지 — 생성자, 정적 팩토리, 빌더

	생성자	정적 팩토리 (아이템 1)	빌더 (아이템 2)
이름	없다	생성 방법에 이름	인자마다 이름
선택 인자	생성자 계단	메서드를 늘린다	메서드 하나씩, 기본값
불변	지킨다	지킨다	지킨다
검사 시점	생성자 안	생성자/팩토리 안	`build()` 한 곳
이 과정에서	`Health(int maxHp)`	`SCV.create()`, `SCV.ofDamaged(1)`	`new UnitStats.Builder(60)…build()`

- 자바빈즈는 이름을 얻는 대신 필수 값 강제와 불변을 잃는다.
- 빌더는 이름과 불변을 함께 얻는 대신 코드가 늘어난다. 인자가 많을 때 그 값을 치른다.

6교시

로그에 남은 건 '닫기 실패'뿐

리플레이를 복사하다 실패했다. 그런데 무엇 때문에 실패했는지가 사라졌다.

준비 코드 (1/2): 메모리로 흉내 낸 리플레이 자원

`ReplayReader.java` — 파일 대신 메모리의 줄 3개를 읽는다. 실패를 원하는 순간에 일으키려고 그렇게 했다.

```
import java.io.IOException;
import java.util.Arrays;
import java.util.List;

class ReplayReader implements AutoCloseable {
    private final String name;
    private final List<String> lines =
            Arrays.asList("00:01 SCV 생산", "00:12 Marine 생산", "00:30 공격 시작");
    private int next = 0;
    private boolean closed = false;

    ReplayReader(String name) {
        this.name = name;
        System.out.println(name + " 열기");
    }

    String readLine() throws IOException {
        return next < lines.size() ? lines.get(next++) : null;
    }

    boolean isClosed() { return closed; }

    @Override
    public void close() throws IOException {
        closed = true;
        System.out.println(name + " 닫기");
    }
}
```

준비 코드 (2/2): 메모리로 흉내 낸 리플레이 자원

ReplayWriter.java — 받은 줄을 메모리 목록에 쌓는다.

```
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

class ReplayWriter implements AutoCloseable {
    private final String name;
    private final List<String> lines = new ArrayList<>();
    private boolean closed = false;

    ReplayWriter(String name) {
        this.name = name;
        System.out.println(name + " 열기");
    }

    void write(String line) throws IOException {
        lines.add(line);
    }

    List<String> getLines() { return new ArrayList<>(lines); }

    boolean isClosed() { return closed; }

    @Override
    public void close() throws IOException {
        closed = true;
        System.out.println(name + " 닫기");
    }
}
```

리플레이 파일을 복사한다: 읽고, 쓰고, 반드시 닫는다

`ReplayCopier.java` 를 만들고 static 메서드로 쓴다. 리플레이 파일은 열었으면 반드시 닫아야 한다. 자원이 하나라면 익숙한 모양이다.

```
import java.io.IOException;

final class ReplayCopier {

    private ReplayCopier() { }   // static 메서드만 모아 두는 클래스

    static int countLines(ReplayReader in) throws IOException {
        try {
            int count = 0;
            while (in.readLine() != null) {
                count++;
            }
            return count;
        } finally {
            in.close();   // 읽다가 실패해도 닫는다
        }
    }
}
```

자원이 둘이면 try-finally가 계단이 된다

`ReplayCopier.java` 에 이어서 쓴다(클래스 안, `countLines` 아래). 읽는 쪽(`in`)과 쓰는 쪽(`out`)을 둘 다 닫으려면 try-finally를 겹쳐야 한다.

```
    static void copy(ReplayReader in, ReplayWriter out) throws IOException {
        try {
            try {
                String line;
                while ((line = in.readLine()) != null) {
                    out.write(line);
                }
            } finally {
                out.close();
            }
        } finally {
            in.close();
        }
    }
```

자원이 하나 늘 때마다 계단이 한 칸 깊어진다. 그래도 닫히기는 다 닫힌다. 정말 괜찮을까?

테스트: 정상이면 모든 줄을 복사하고 둘 다 닫는다

ReplayCopierTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.io.IOException;
import java.util.Arrays;

import static org.junit.jupiter.api.Assertions.*;

class ReplayCopierTest {

    @Test
    @DisplayName("정상이면 모든 줄을 복사하고 두 자원을 닫는다")
    void copiesAllLines() throws IOException {
        ReplayReader in = new ReplayReader("game.rep");
        ReplayWriter out = new ReplayWriter("copy.rep");

        ReplayCopier.copy(in, out);

        assertEquals(Arrays.asList("00:01 SCV 생산", "00:12 Marine 생산", "00:30 공격 시작"),
                out.getLines());
        assertTrue(in.isClosed());
        assertTrue(out.isClosed());
    }
}
```

```
SUCCESSFUL 정상이면 모든 줄을 복사하고 두 자원을 닫는다
```

준비 코드: 쓰기도 닫기도 실패하는 기록기

FaultyReplayWriter.java — 실패 상황을 테스트로 재현하려고 만든다. `write`에서 「기록 실패」, `close`에서 「닫기 실패」를 던진다.

```
import java.io.IOException;

class FaultyReplayWriter extends ReplayWriter {   // 테스트용: 쓰기도 닫기도 실패한다

    FaultyReplayWriter(String name) {
        super(name);
    }

    @Override
    void write(String line) throws IOException {
        throw new IOException("기록 실패");
    }

    @Override
    public void close() throws IOException {
        super.close();   // 닫힘 표시와 출력은 그대로 한 뒤
        throw new IOException("닫기 실패");
    }
}
```

기록 중에 실패하고 닫기도 실패하면?

`ReplayCopierTest.java` 에 이어서 두 테스트를 클래스 안에 추가하고 실행한다.

```
    @Test
    @DisplayName("기록이 실패해도 두 자원은 모두 닫힌다")
    void closesBothEvenWhenWriteFails() {
        ReplayReader in = new ReplayReader("game.rep");
        ReplayWriter out = new FaultyReplayWriter("copy.rep");

        assertThrows(IOException.class, () -> ReplayCopier.copy(in, out));

        assertTrue(in.isClosed());
        assertTrue(out.isClosed());
    }

    @Test
    @DisplayName("기록 중 실패하면 호출부는 '기록 실패' 예외를 받는다")
    void keepsWriteFailure() {
        ReplayReader in = new ReplayReader("game.rep");
        ReplayWriter out = new FaultyReplayWriter("copy.rep");

        IOException thrown = assertThrows(IOException.class, () -> ReplayCopier.copy(in, out));

        assertEquals("기록 실패", thrown.getMessage());
    }
```

```
SUCCESSFUL 기록이 실패해도 두 자원은 모두 닫힌다
FAILED     기록 중 실패하면 호출부는 '기록 실패' 예외를 받는다  expected: <기록 실패> but was: <닫기 실패>
```

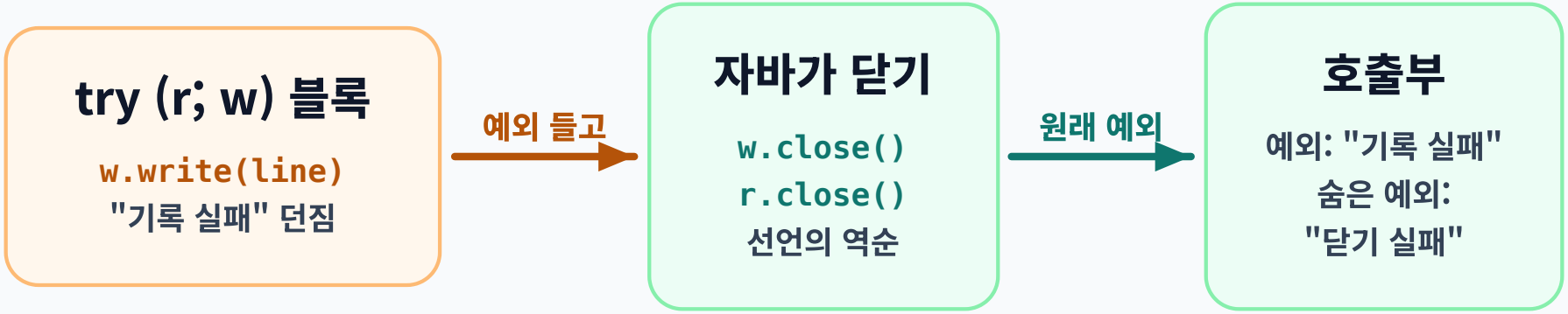
원래 예외는 어디로 갔나



finally에서 던진 예외가 try 블록의 예외를 **덮어쓴다**. 원래 원인인 「기록 실패」를 지키면서 둘 다 닫으려면 어떻게 해야 할까?

> 현업 장면: finally에서 JDBC `Connection`을 닫다가 로그에 닫기 예외만 남고 진짜 SQL 오류가 사라졌다면 바로 이 장면이다.

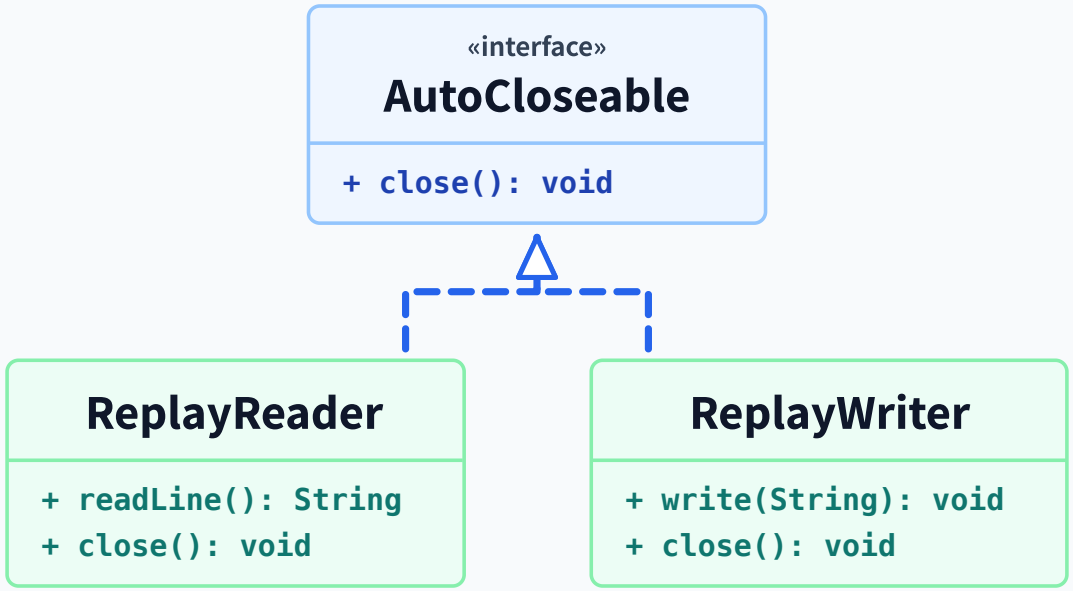
자원을 선언하면 자바가 대신 닫는다



자원을 `try` 괄호 안에 선언하면, 블록이 끝날 때 자바가 **선언의 역순으로** 닫는다. 닫다가 난 예외는 버려지지 않고 원래 예외 안에 **숨은 예외(suppressed)**로 붙는다. 이 문법을 **try-with-resources**라 한다.

> 이것이 이펙티브 자바 아이템 9: try-finally보다는 try-with-resources를 사용하라.

AutoCloseable: '닫을 수 있다'는 역할



`try (...)` 괄호에는 `AutoCloseable`을 구현한 객체만 들어간다. 1교시의 역할 인터페이스처럼 「닫을 수 있다」는 약속 하나다. `InputStream`, `Connection`, `ResultSet`도 이미 이 약속을 지킨다.

try-with-resources 최소 구현

자원 쪽은 `AutoCloseable`을 구현한다. 준비 코드의 두 자원은 이미 그렇게 만들었다(발췌).

```
class ReplayReader implements AutoCloseable {
    @Override
    public void close() throws IOException {
        closed = true;
        System.out.println(name + " 닫기");
    }
}
```

쓰는 쪽: ReplayCopier.java의 `copy()`를 이것으로 바꾼다. 괄호 안에 자원을 선언하고, finally를 지운다.

```
    static void copy(ReplayReader in, ReplayWriter out) throws IOException {
        try (ReplayReader r = in; ReplayWriter w = out) {
            String line;
            while ((line = r.readLine()) != null) {
                w.write(line);
            }
        }
    }
```

테스트: 원래 예외는 「기록 실패」, 숨은 예외는 「닫기 실패」

`ReplayCopierTest.java` 에 이어서 마지막 테스트를 추가하고, 네 건을 모두 다시 실행한다.

```
    @Test
    @DisplayName("닫기 실패는 숨은 예외로 함께 남는다")
    void keepsCloseFailureAsSuppressed() {
        ReplayReader in = new ReplayReader("game.rep");
        ReplayWriter out = new FaultyReplayWriter("copy.rep");

        IOException thrown = assertThrows(IOException.class, () -> ReplayCopier.copy(in, out));

        assertEquals(1, thrown.getSuppressed().length);
        assertEquals("닫기 실패", thrown.getSuppressed()[0].getMessage());
    }
```

`ReplayCopierTest`	try-finally	try-with-resources
정상이면 모든 줄을 복사하고 두 자원을 닫는다	통과	통과
기록이 실패해도 두 자원은 모두 닫힌다	통과	통과
호출부는 '기록 실패' 예외를 받는다	**실패**	통과
닫기 실패는 숨은 예외로 함께 남는다	**실패**	통과

두 자원은 어떤 순서로 닫힐까? (퀴즈)

퀴즈 CloseOrderDemo.java — 수업용 자원은 만들 때 `열기`, 닫을 때 `닫기`를 출력한다.

```
import java.io.IOException;

public class CloseOrderDemo {

    public static void main(String[] args) throws IOException {
        try (ReplayReader r = new ReplayReader("game.rep");
             ReplayWriter w = new ReplayWriter("copy.rep")) {
            w.write(r.readLine());
            System.out.println("한 줄 복사");
        }
    }
}
```

마지막 두 줄은?

1. `game.rep 닫기` → `copy.rep 닫기` (선언한 순서)
2. `copy.rep 닫기` → `game.rep 닫기` (선언의 역순)
3. 실행할 때마다 다르다

풀이: 선언의 역순으로 닫힌다

정답

정답은 2번이다. `CloseOrderDemo` 실행 결과:

```
game.rep 열기
copy.rep 열기
한 줄 복사
copy.rep 닫기
game.rep 닫기
```

> 나중에 연 자원이 먼저 연 자원에 기대고 있을 수 있다(쓰는 쪽이 읽은 줄을 받는다). 그래서 나중에 연 것부터 닫는다. 손으로 쓴 계단식 finally와 같은 순서를 자바가 지켜 준다.

catch와 finally도 함께 쓸 수 있다

실패를 그 자리에서 처리하고 싶으면 catch와 finally를 그대로 붙인다. `ReplayCopier.java`에 이어서 쓴다(클래스 안, `copy` 아래).

```
    static boolean tryCopy(ReplayReader in, ReplayWriter out) {
        try (ReplayReader r = in; ReplayWriter w = out) {
            String line;
            while ((line = r.readLine()) != null) {
                w.write(line);
            }
            return true;
        } catch (IOException e) {
            System.out.println("복사 실패: " + e.getMessage()
                    + " (숨은 예외 " + e.getSuppressed().length + "개)");
            return false;
        } finally {
            System.out.println("복사 시도 끝");
        }
    }
```

catch와 finally는 **자원이 모두 닫힌 뒤에** 실행된다. 정말 그런지 출력으로 확인해 보자.

catch에 도착했을 때 자원은 이미 닫혀 있다

CatchFinallyDemo.java — `FaultyReplayWriter`로 `tryCopy`를 부른다.

```
public class CatchFinallyDemo {

    public static void main(String[] args) {
        boolean copied = ReplayCopier.tryCopy(
                new ReplayReader("game.rep"), new FaultyReplayWriter("copy.rep"));
        System.out.println("결과: " + copied);
    }
}
```

```
game.rep 열기
copy.rep 열기
copy.rep 닫기
game.rep 닫기
복사 실패: 기록 실패 (숨은 예외 1개)
복사 시도 끝
결과: false
```

> catch가 받은 예외는 원래 원인인 「기록 실패」이고, 「닫기 실패」는 그 안에 숨은 예외로 함께 들어 있다.

실무 형태: 파일 복사를 try-with-resources로

`FileInputStream`, `FileOutputStream`도 `AutoCloseable`이라 같은 모양이 된다(발췌, 따라 치지 않아도 된다).

```
static void copy(String src, String dst) throws IOException {
    try (InputStream in = new FileInputStream(src);
         OutputStream out = new FileOutputStream(dst)) {
        byte[] buf = new byte[1024];
        int n;
        while ((n = in.read(buf)) >= 0) {
            out.write(buf, 0, n);
        }
    }
}
```

> JDBC도 같다. `Connection`, `PreparedStatement`, `ResultSet`을 괄호 안에 차례로 선언하면 finally의 `close()` 줄이 모두 사라진다.

Java 8에서는 try 괄호 안에서 변수를 선언해야 한다

이미 있는 변수를 괄호에 그대로 넣는 문법은 Java 9부터다(발췌).

```
static void copy(ReplayReader in, ReplayWriter out) throws IOException {
    try (in; out) {   // 컴파일 에러(Java 8): Java 9부터 가능한 문법
        // ...
    }
}
```

Java 8에서는 새 변수를 선언하며 받아 둔다.

```
try (ReplayReader r = in; ReplayWriter w = out) {
```

> 인터넷 예제의 `try (in)`을 그대로 옮기면 Java 8 프로젝트에서는 빌드가 깨진다.

레거시 ReplayArchiver를 try-with-resources로 바꾼다 (실습)

실습 파일: `ReplayArchiver.java` · 테스트: `ReplayArchiverTest.java` (다음 장). 레거시 코드를 먼저 그대로 만든다.

```
import java.io.IOException;
import java.util.List;

final class ReplayArchiver {

    private ReplayArchiver() { }

    static void archive(ReplayWriter out, List<String> lines) throws IOException {
        try {
            for (String line : lines) {
                out.write(line);
            }
        } finally {
            out.close();
        }
    }
}
```

1. 다음 장의 테스트를 만들어 실행한다. `but was: <닫기 실패>`로 1건 실패한다.
2. finally를 지우고 `out`을 try 괄호 안의 새 변수로 받는다.
3. 다시 실행해 2건 통과를 확인한다(원래 예외 「기록 실패」, `getSuppressed().length == 1`).

실습용 테스트: ReplayArchiverTest

ReplayArchiverTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.io.IOException;
import java.util.Arrays;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class ReplayArchiverTest {

    @Test
    @DisplayName("정상이면 모든 줄을 기록하고 닫는다")
    void archivesAllLines() throws IOException {
        ReplayWriter out = new ReplayWriter("archive.rep");
        List<String> lines = Arrays.asList("00:01 SCV 생산", "00:12 Marine 생산");
        ReplayArchiver.archive(out, lines);
        assertEquals(lines, out.getLines());
        assertTrue(out.isClosed());
    }

    @Test
    @DisplayName("기록 실패 예외가 보존되고 닫기 실패는 숨은 예외로 남는다")
    void keepsWriteFailure() {
        ReplayWriter out = new FaultyReplayWriter("archive.rep");
        List<String> lines = Arrays.asList("00:01 SCV 생산");
        IOException thrown = assertThrows(IOException.class, () -> ReplayArchiver.archive(out, lines));
        assertEquals("기록 실패", thrown.getMessage());
        assertEquals(1, thrown.getSuppressed().length);
    }
}
```

실습 풀이: 괄호 안에 선언하고 finally를 지운다

정답

ReplayArchiver.java 의 `archive()` 를 이것으로 바꾼다.

```
    static void archive(ReplayWriter out, List<String> lines) throws IOException {
        try (ReplayWriter w = out) {
            for (String line : lines) {
                w.write(line);
            }
        }
    }
```

```
SUCCESSFUL 정상이면 모든 줄을 기록하고 닫는다
SUCCESSFUL 기록 실패 예외가 보존되고 닫기 실패는 숨은 예외로 남는다
```

정리: 자원 클래스를 만들면 AutoCloseable을 구현한다

상황	이렇게 한다
닫아야 하는 자원을 쓴다	`try (자원 선언) { ... }` — finally에서 닫지 않는다
자원이 여러 개다	괄호 안에 `;`로 이어 선언한다. 역순으로 닫힌다
닫기 실패도 알고 싶다	`e.getSuppressed()`로 꺼낸다
자원 클래스를 직접 만든다	`AutoCloseable`을 구현해 `close()`를 약속한다
Java 8 프로젝트다	괄호 안에서 새 변수로 선언한다(`try (in)` 불가)

> 이펙티브 자바 아이템 9: try-finally보다는 try-with-resources를 사용하라. 원래 예외를 지키는 것이 핵심이다.

7교시

같은 좌표, 다른 객체

(10, 20)을 넣고 (10, 20)을 찾았다. 그런데 없다고 한다.

SCV가 지나간 좌표를 기록한다: List<Position>

4교시에 만든 불변 `Position`을 그대로 쓴다(발췌: 4교시 Position.java 그대로, 다시 치지 않는다). 새 요구: 지나간 좌표를 기록하고, 지나간 적 있는지 묻는다.

```
public final class Position {                // 규칙 1: 상속으로 못 바꾸게 final
    private final int x;                     // 규칙 2: 모든 필드를 private final로
    private final int y;

    public Position(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }          // 규칙 3: setter를 두지 않는다
    public int getY() { return y; }

    public Position move(int dx, int dy) {   // 규칙 4: 바뀐 값은 새 객체로
        return new Position(x + dx, y + dy);
    }
}
```

```
List<Position> visited = new ArrayList<>();
visited.add(new Position(10, 20));           // 집결지를 지나갔다
visited.contains(new Position(10, 20));      // 지나간 적 있나?
```

(10, 20)을 넣고 (10, 20)을 찾으면 false

PositionEqualityTest.java 새 파일. 기록할 때와 물을 때 각각 `new Position(10, 20)`을 만든다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertFalse;
import static org.junit.jupiter.api.Assertions.assertTrue;

class PositionEqualityTest {

    @Test
    @DisplayName("지나간 좌표 (10, 20)을 다시 물으면 List에서 찾는다")
    void listContainsSamePosition() {
        List<Position> visited = new ArrayList<>();
        visited.add(new Position(10, 20));

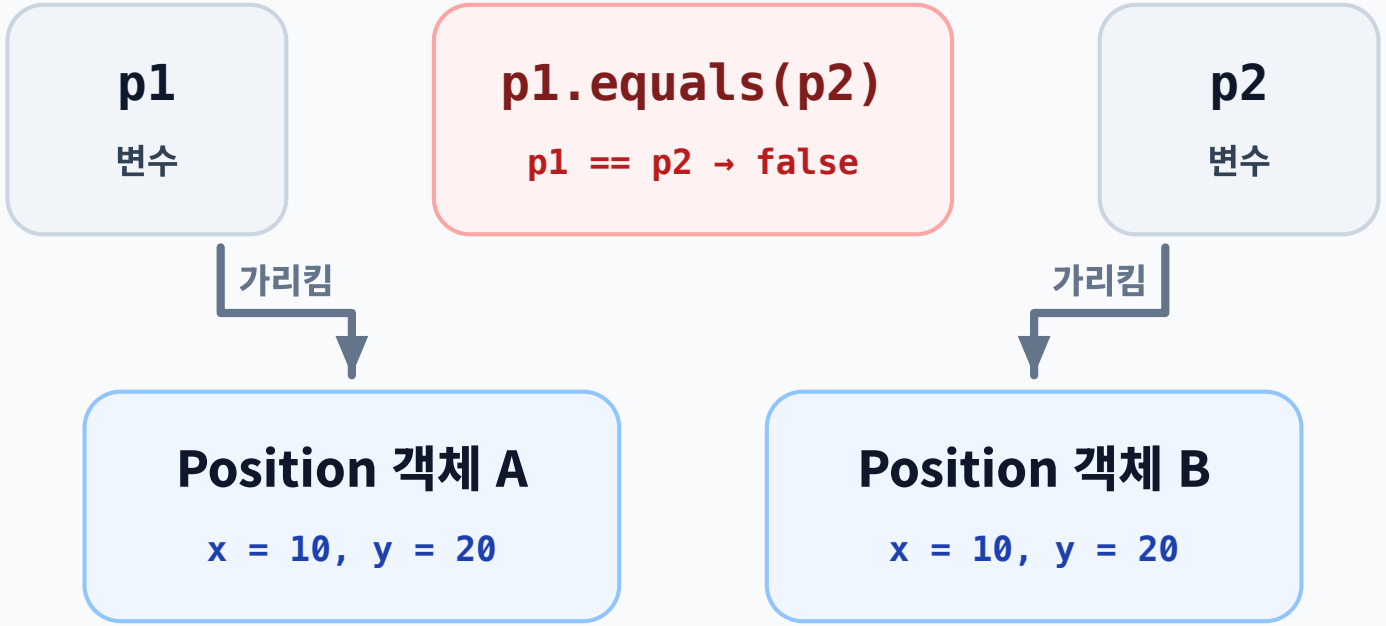
        boolean found = visited.contains(new Position(10, 20));

        assertTrue(found);
    }
}
```

```
FAILED  expected: <true> but was: <false>
```

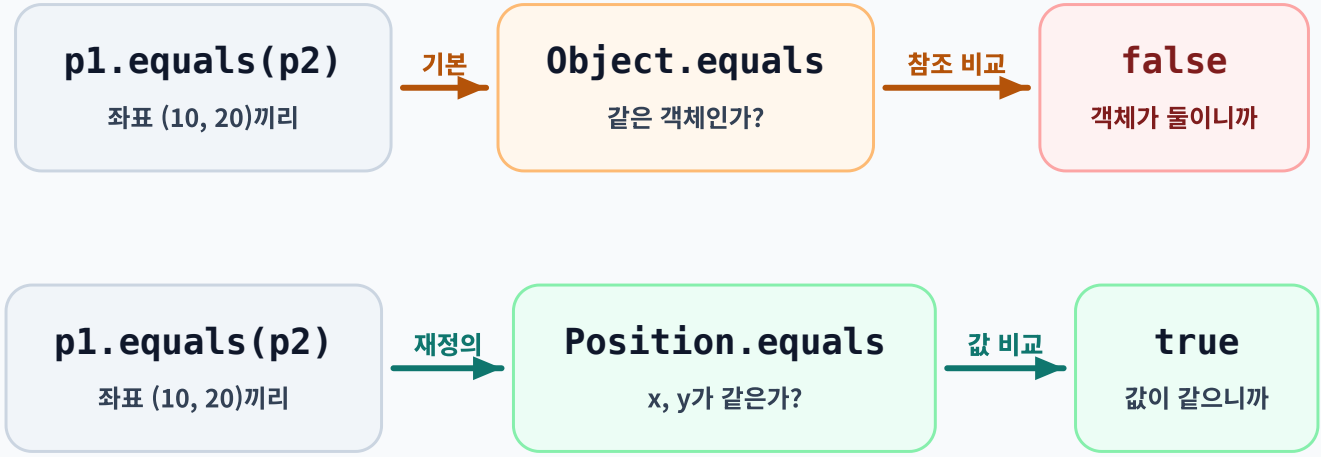
`List.contains`는 원소마다 `equals`를 부른다. `Position`엔 `equals`가 없는데, 누구의 것이 불렸을까?

기본 equals는 '같은 객체인가'만 묻는다



`equals`를 재정의하지 않으면 `Object`의 `return this == obj;`가 불린다. 좌표가 같아도 **객체가 둘이면** 다르다고 답한다.

값이 같으면 같은 것으로 정의한다



`Position`은 「어떤 객체인가」보다 「어떤 값인가」가 중요한 **값 객체**다. 값 객체는 `equals`를 재정의해 값으로 비교하게 한다.

> 이것이 이펙티브 자바 아이템 10: equals는 일반 규약을 지켜 재정의하라. `String`, `Integer`, `LocalDate`가 이미 이렇게 동작한다.

equals 최소 구현

세 단계다: 같은 객체면 바로 true, 타입이 다르면 false, 같은 타입이면 필드를 비교한다. `Position.java`의 클래스 안, `move` 아래에 추가한다.

```java
@Override
public boolean equals(Object o) {
    if (this == o) {
        return true;                        // 같은 객체: 비교할 필요 없다
    }
    if (!(o instanceof Position)) {
        return false;                       // null이거나 다른 타입
    }
    Position other = (Position) o;
    return x == other.x && y == other.y;    // 값 비교
}
```

> 매개변수 타입은 반드시 `Object`다. `equals(Position o)`로 쓰면 재정의가 아니라 오버로딩이 되어 컬렉션은 여전히 `Object`의 것을 부른다. `@Override`를 붙이면 컴파일러가 이 실수를 잡는다.

테스트 통과: List.contains가 true

`PositionEqualityTest.java`에 이어서 두 번째 테스트를 클래스 안에 추가하고, 두 건을 실행한다.

```
SUCCESSFUL 지나간 좌표 (10, 20)을 다시 물으면 List에서 찾는다
SUCCESSFUL 좌표가 다르면 같지 않고, null과도 같지 않다
```

```java
@Test
@DisplayName("좌표가 다르면 같지 않고, null과도 같지 않다")
void differentOrNullIsNotEqual() {
    Position p = new Position(10, 20);

    assertFalse(p.equals(new Position(99, 20)));
    assertFalse(p.equals(null));
}
```

`instanceof`는 `null`이면 false라서 null 검사를 따로 쓰지 않아도 된다. `Position`은 `final`이라 하위 클래스가 끼어들 걱정도 없다.

equals가 지켜야 할 다섯 약속

`equals`는 컬렉션, 캐시, 정렬이 모두 믿고 부르는 메서드다. 그래서 다섯 가지를 약속해야 한다.

약속	뜻 (null이 아닌 x, y, z)	깨지면 생기는 일
반사성	`x.equals(x)`는 true	넣은 것을 못 찾는다
대칭성	`x.equals(y)` ⇔ `y.equals(x)`	누가 묻느냐에 따라 답이 바뀐다
추이성	x = y, y = z면 x = z	하위 클래스가 비교 기준을 바꾼다
일관성	값이 그대로면 늘 같은 답	시각·네트워크 같은 바깥 값에 흔들린다
null 아님	`x.equals(null)`은 false	예외가 터진다

> 방금 만든 `Position.equals`는 다섯 개를 모두 지킨다. 같은 타입끼리만, 바뀌지 않는 필드로만 비교하기 때문이다.

이 equals는 어느 약속을 깰까? (퀴즈)

UnitName.java — 유닛 이름을 대소문자 구분 없이 비교하고 싶다. 친절하게 `String`과도 비교해 준다.

```
public final class UnitName {
    private final String value;

    public UnitName(String value) {
        this.value = value;
    }

    @Override
    public boolean equals(Object o) {
        if (o instanceof UnitName) {
            return value.equalsIgnoreCase(((UnitName) o).value);
        }
        if (o instanceof String) {   // String과도 같다고 답한다
            return value.equalsIgnoreCase((String) o);
        }
        return false;
    }
}
```

1. **반사성** — 객체는 자기 자신과 같다
2. **대칭성** — A가 B와 같으면, B도 A와 같다
3. **추이성** — A가 B와 같고 B가 C와 같으면, A도 C와 같다
4. **null 비교** — 객체는 `null`과 같지 않다

풀이: 대칭성 — 다른 타입과는 같다고 하지 않는다

정답 2번이다. `UnitNameDemo.java`로 양쪽에서 물어본다.

```
import java.util.ArrayList;
import java.util.List;

public class UnitNameDemo {
    public static void main(String[] args) {
        UnitName name = new UnitName("Marine");
        String raw = "marine";
        System.out.println("name.equals(raw) = " + name.equals(raw));
        System.out.println("raw.equals(name) = " + raw.equals(name));
        List<UnitName> names = new ArrayList<>();
        names.add(name);
        System.out.println("names.contains(raw) = " + names.contains(raw));
    }
}
```

```
name.equals(raw) = true
raw.equals(name) = false
names.contains(raw) = false
```

> `String`은 `UnitName`을 모른다. 고치는 법: `String` 분기를 지우고 같은 타입끼리만 비교한다.

List를 HashSet으로 바꾸면 다시 false

지나간 좌표가 많아져서 중복 없이, 빨리 찾으려고 `HashSet`으로 바꿨다. `equals`는 그대로다. `PositionHashTest.java` 새 파일을 만든다(뒤에서 쓸 import까지 미리 넣었다).

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class PositionHashTest {

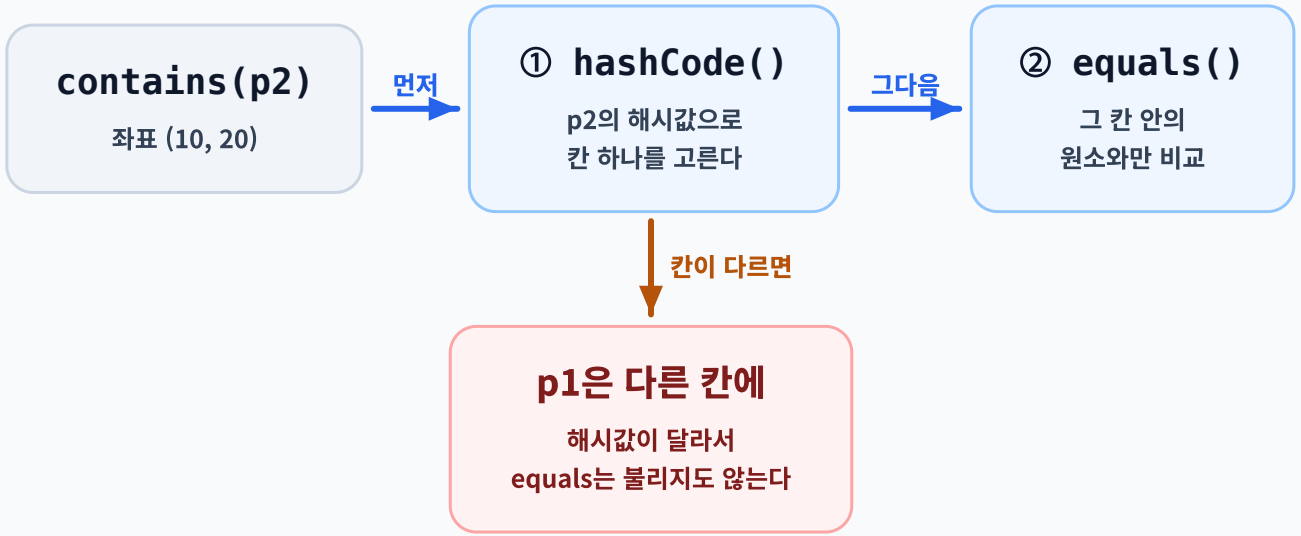
    @Test
    @DisplayName("지나간 좌표 (10, 20)을 다시 물으면 HashSet에서 찾는다")
    void hashSetContainsSamePosition() {
        Set<Position> visited = new HashSet<>();
        visited.add(new Position(10, 20));

        boolean found = visited.contains(new Position(10, 20));

        assertTrue(found);
    }
}
```

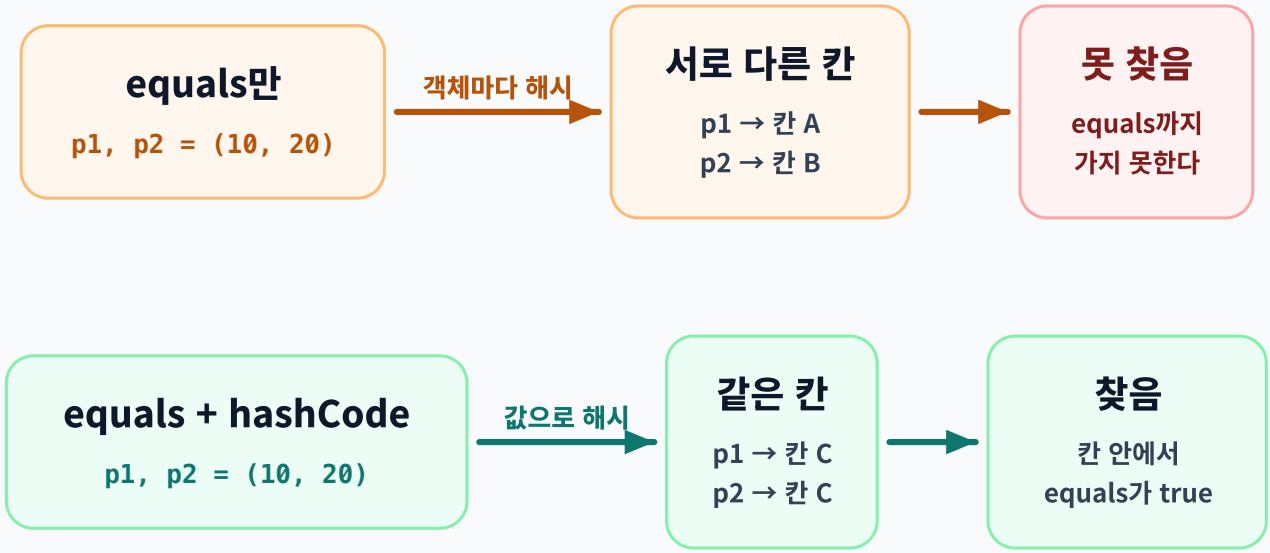
```
FAILED  expected: <true> but was: <false>      (같은 equals로 List 테스트는 통과)
```

HashSet은 먼저 해시 칸을 찾고, 그 칸 안에서만 equals를 부른다



`HashSet` · `HashMap`은 모든 원소와 비교하지 않는다. `hashCode()`로 칸을 고른 뒤 그 칸만 본다. `hashCode`를 재정의하지 않았으니 p1과 p2는 객체마다 다른 값을 받아 다른 칸에 있다.

equals가 같으면 hashCode도 같아야 한다



같은 값이면 같은 칸에 들어가야 `equals` 까지 간다. **hashCode도 같은 필드로 재정의한다.**

> 이것이 이펙티브 자바 아이템 11: equals를 재정의하려거든 hashCode도 재정의하라.
> `javac -Xlint` 도 「overrides equals, but ... hashCode」 경고로 알려 준다.

hashCode 최소 구현: Objects.hash

`equals`에 쓴 필드를 그대로 `Objects.hash`에 넘긴다. Position.java의 `equals` 아래에 추가한다(`import`는 다음 장의 완성본에).

```
@Override
public int hashCode() {
    return Objects.hash(x, y);
}
```

hashCode가 지킬 것	`Position`
equals가 true면 hashCode가 같다	같은 필드 `x, y`로 계산
값이 그대로면 몇 번 불러도 같다	필드가 `final`
equals가 false여도 같을 수는 있다(다르면 더 빠르다)	값이 다르면 대부분 다른 값

> `Objects.hash`는 부를 때마다 배열을 만든다. 아주 자주 불리는 곳이면 `31 * Integer.hashCode(x) + Integer.hashCode(y)`처럼 직접 계산한다.

Position.java 완성: 불변 + equals + hashCode

Position.java 전체 — 맨 위 `import java.util.Objects;`를 확인한다. 왼쪽 다음에 오른쪽이 이어진다. Day 2도 이 파일을 그대로 쓴다.

```
import java.util.Objects;

public final class Position {                // 규칙 1: 상속으로 못 바꾸게 final
    private final int x;                     // 규칙 2: 모든 필드를 private final로
    private final int y;

    public Position(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() { return x; }          // 규칙 3: setter를 두지 않는다
    public int getY() { return y; }

    public Position move(int dx, int dy) {   // 규칙 4: 바뀐 값은 새 객체로
        return new Position(x + dx, y + dy);
    }
```

```
    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;                     // 같은 객체: 비교할 필요 없다
        }
        if (!(o instanceof Position)) {
            return false;                    // null이거나 다른 타입
        }
        Position other = (Position) o;
        return x == other.x && y == other.y;   // 값 비교
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```

테스트 통과: HashSet.contains, HashMap 키 조회

PositionHashTest.java 에 이어서 두 테스트를 클래스 안에 추가하고 세 건을 모두 실행한다. `HashMap` 키 조회도 같은 원리다.

```
@Test
@DisplayName("같은 좌표를 HashSet에 두 번 넣으면 크기 1")
void hashSetKeepsOneOfSamePosition() {
    Set<Position> visited = new HashSet<>();

    visited.add(new Position(10, 20));
    visited.add(new Position(10, 20));

    assertEquals(1, visited.size());
}

@Test
@DisplayName("같은 좌표로 HashMap에 표시한 이름을 꺼낸다")
void hashMapFindsBySamePosition() {
    Map<Position, String> marks = new HashMap<>();
    marks.put(new Position(10, 20), "집결지");

    String mark = marks.get(new Position(10, 20));

    assertEquals("집결지", mark);
}
```

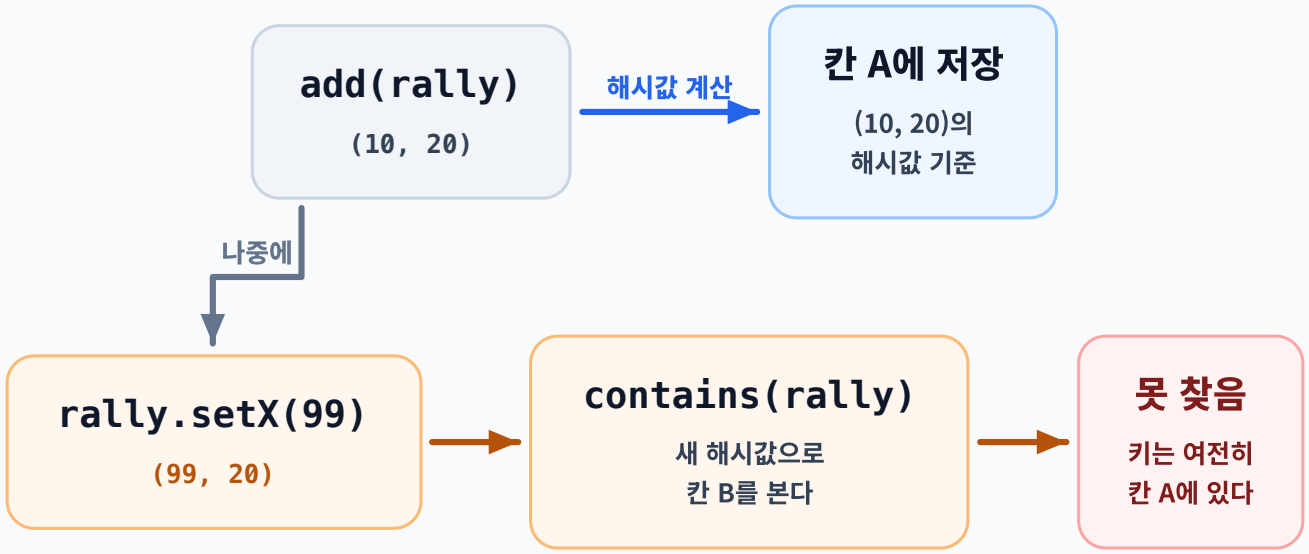
세 단계를 표로: 무엇을 재정의했나에 따라

`PositionEqualityTest`, `PositionHashTest`를 단계마다 돌린 결과다. 4교시 상태 → equals 추가 → hashCode 추가.

테스트	equals 없음	equals만	equals + hashCode
`List.contains`	**실패**	통과	통과
`HashSet.contains`	**실패**	**실패**	통과
`HashSet` 두 번 `add` → 크기 1	**실패** (2)	**실패** (2)	통과
`HashMap.get`	**실패**	**실패** (`null`)	통과

> 현업 장면: VO를 `HashSet`에 넣었는데 중복 제거가 안 되거나, `HashMap` 키로 썼는데 `get`이 `null`을 돌려준 적이 있다면 바로 이 장면이다.

불변이어야 하는 이유 하나 더: 바뀐 키는 못 찾는다



4교시의 가변 `MutablePosition`에 equals/hashCode를 붙여 `HashSet`에 넣고 `setX(99)`를 부르면, `size()`는 1인데 `contains(rally)`는 false다(`MutableKeyTest`). 키로 쓰는 값 객체는 불변이어야 한다.

확인: 바뀐 키는 들어 있는데도 못 찾는다

왼쪽: 4교시의 `MutablePosition.java` 맨 위에 `import java.util.Objects;`를 넣고, `setY` 아래에 이것을 붙인다. 오른쪽: `MutableKeyTest.java` 새 파일.

```
@Override
public boolean equals(Object o) {
    if (this == o) {
        return true;
    }
    if (!(o instanceof MutablePosition)) {
        return false;
    }
    MutablePosition other = (MutablePosition) o;
    return x == other.x && y == other.y;
}

@Override
public int hashCode() {
    return Objects.hash(x, y);
}
```

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.HashSet;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;

class MutableKeyTest {

    @Test
    @DisplayName("HashSet에 넣은 뒤 좌표를 바꾼 키는 들어 있는데도 찾지 못한다")
    void mutatedKeyIsLost() {
        Set<MutablePosition> visited = new HashSet<>();
        MutablePosition rally = new MutablePosition(10, 20);
        visited.add(rally);

        rally.setX(99);

        assertEquals(1, visited.size());
        assertFalse(visited.contains(rally));
    }
}
```

Health에도 equals와 hashCode를 (실습)

실습

파일 수정: 4교시에 불변으로 바꾼 `Health.java` — 아래는 발췌다.

```
public final class Health {
    private final int currentHp;
    private final int maxHp;
    // 생성자, restore, damage, getHp, getMaxHp는 4교시 그대로
    // 실습: equals와 hashCode를 currentHp, maxHp로 재정의한다
}
```

1. 다음 장의 `HealthEqualityTest`를 만들어 실행한다. 2건 실패: `expected: <Health@…> but was: <Health@…>` (주소는 실행마다 다름), `expected: <1> but was: <2>`
2. `Position`과 같은 모양으로 equals를, 같은 두 필드로 `Objects.hash`를 쓴다(`import java.util.Objects;` 추가).
3. 다시 실행해 2건 통과를 확인한다.

이제 체력도 `assertEquals`로 값끼리 비교할 수 있다.

실습용 테스트: HealthEqualityTest

HealthEqualityTest.java

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.HashSet;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.assertEquals;

class HealthEqualityTest {

    @Test
    @DisplayName("체력 30/60 두 개는 같다")
    void sameHpAndMaxAreEqual() {
        Health a = new Health(30, 60);
        Health b = new Health(30, 60);
        assertEquals(a, b);
    }

    @Test
    @DisplayName("HashSet에 두 번 넣으면 크기 1")
    void hashSetKeepsOne() {
        Set<Health> set = new HashSet<>();
        set.add(new Health(30, 60));
        set.add(new Health(30, 60));
        assertEquals(1, set.size());
    }
}
```

실습 풀이: 같은 두 필드로 equals와 hashCode

정답 Health.java 의 `getMaxHp` 아래에 붙인다(맨 위에 `import java.util.Objects;`).

```
@Override
public boolean equals(Object o) {
    if (this == o) {
        return true;
    }
    if (!(o instanceof Health)) {
        return false;
    }
    Health other = (Health) o;
    return currentHp == other.currentHp && maxHp == other.maxHp;
}

@Override
public int hashCode() {
    return Objects.hash(currentHp, maxHp);
}
```

```
SUCCESSFUL 체력 30/60 두 개는 같다
SUCCESSFUL HashSet에 두 번 넣으면 크기 1
```

반대로 hashCode만 재정의하면 무엇이 깨질까? (퀴즈)

퀴즈

equals만 재정의했을 때는 `HashSet` · `HashMap`이 깨졌다. 이번엔 equals 없이 hashCode만 둔 좌표 클래스 `HashOnlyPosition`을 따로 만든다(`Position`은 그대로 둔다). 세 칸을 예측해 보자.

재정의한 것	`List.contains`	`HashSet.contains`	`HashSet`에 같은 좌표 두 번 `add` 후 `size()`
hashCode만	?	?	?

힌트: `List`는 equals만 부른다. `HashSet`은 hashCode로 칸을 찾고, 그 칸에서 equals를 부른다.

풀이: 같은 칸에 와도 equals가 다르다고 하면 못 찾는다

정답

재정의한 것	`List.contains`	`HashSet.contains`	두 번 `add` 후 `size()`
hashCode만	**false**	**false** (같은 칸, equals가 false)	**2**
(앞에서 본) equals만	true	**false** (칸이 다르다)	**2**

HashOnlyPosition.java — 확인용 좌표. hashCode만 있고 equals는 `Object`의 것이다.

```
public final class HashOnlyPosition {
    private final int x;
    private final int y;

    public HashOnlyPosition(int x, int y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public int hashCode() {
        return 31 * x + y;
    }
}
```

확인: HashCodeOnlyTest

HashCodeOnlyTest.java — 두 건 모두 통과한다. 표의 「hashCode만」 행이 그대로 나온다.

```java
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

import static org.junit.jupiter.api.Assertions.*;

class HashCodeOnlyTest {

    @Test
    @DisplayName("hashCode만 재정의하면 같은 칸에 가도 equals가 false라 중복이 쌓인다")
    void sameBucketButNotEqual() {
        Set<HashOnlyPosition> visited = new HashSet<>();
        visited.add(new HashOnlyPosition(10, 20));
        visited.add(new HashOnlyPosition(10, 20));
        assertEquals(2, visited.size());
        assertFalse(visited.contains(new HashOnlyPosition(10, 20)));
    }

    @Test
    @DisplayName("hashCode만 재정의하면 List.contains도 같은 좌표를 찾지 못한다")
    void listStillUsesIdentity() {
        List<HashOnlyPosition> visited = new ArrayList<>();
        visited.add(new HashOnlyPosition(10, 20));
        assertFalse(visited.contains(new HashOnlyPosition(10, 20)));
    }
}
```

정리: 값 객체 체크리스트 — 불변 + equals + hashCode

같은 칸에 와도 equals가 「다르다」고 하면 못 찾고, equals가 같다고 해도 칸이 다르면 거기까지 가지 못한다. 둘은 **한 쌍**이다. 값 객체마다 세 가지를 확인한다.

1. **불변**: `final` 클래스, `private final` 필드 — 키로 넣은 뒤 바뀌지 않는다
2. **equals**: `Object`를 받고, 같은 타입끼리만, 값 필드로 비교한다
3. **hashCode**: equals에 쓴 필드와 똑같은 필드로 `Objects.hash`

8교시

게임에서 현업 도메인으로: 도서 목록

오늘 SCV로 익힌 값 객체 기법을 도서 주문 시스템의 `Book` 에 옮긴다.

오늘 만든 값 객체를 현업 도메인에 옮긴다: Book

도서 주문 시스템에 이런 클래스가 있다. 현업 코드에서 매일 보는 모양이다.

Book.java

```
public class Book {
    private String isbn;
    private String title;

    public Book(String isbn, String title) {
        this.isbn = isbn;
        this.title = title;
    }

    public String getIsbn() { return isbn; }
    public String getTitle() { return title; }
}
```

- 이번 교시에 이 `Book`을 오늘 배운 값 객체로 완성한다. Day 2 주문 예제가 이 `Book`을 쓴다.
- 첫 요구사항: **도서 목록을 ISBN 순서로 보여 달라.**

List<String>은 정렬되는데 List<Book>은 컴파일되지 않는다

`Collections.sort(names)`는 문자열 목록이면 잘 된다. 책 목록에 똑같이 써 보자.

```
List<Book> books = new ArrayList<>();
books.add(new Book("978-3", "도메인 주도 설계"));
books.add(new Book("978-1", "객체지향"));
books.add(new Book("978-2", "자바 설계"));

Collections.sort(books);   // 컴파일 에러
```

```
error: no suitable method found for sort(List<Book>)
    (inference variable T#1 has incompatible bounds
      equality constraints: Book
      upper bounds: Comparable<? super T#1>)
```

`String`은 두 문자열 중 무엇이 앞인지 스스로 안다. `Book`은 ISBN을 가졌지만 "ISBN으로 줄 서라"고 말한 적이 없다.

정렬할 때마다 익명 Comparator를 쓰게 된다

정렬 순서를 밖에서 알려 주면 컴파일은 된다. 화면마다 같은 익명 클래스를 복사한다. BookSortTest.java

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.TreeSet;

import static org.junit.jupiter.api.Assertions.assertEquals;

class BookSortTest {

    private List<Book> books() {
        List<Book> books = new ArrayList<>();
        books.add(new Book("978-3", "도메인 주도 설계"));
        books.add(new Book("978-1", "객체지향"));
        books.add(new Book("978-2", "자바 설계"));
        return books;
    }
```

```
    @Test
    @DisplayName("주문 화면: 익명 Comparator로 ISBN 순 정렬")
    void orderScreenSortsByIsbn() {
        List<Book> books = books();

        Collections.sort(books, new Comparator<Book>() {
            @Override
            public int compare(Book a, Book b) {
                return a.getIsbn().compareTo(b.getIsbn());
            }
        });

        assertEquals("978-1", books.get(0).getIsbn());
    }

    @Test
    @DisplayName("재고 화면: 복사한 Comparator도 ISBN 순이어야 한다")
    void stockScreenSortsByIsbn() {
        List<Book> books = books();

        Collections.sort(books, new Comparator<Book>() {
            @Override
            public int compare(Book a, Book b) {
                return b.getIsbn().compareTo(a.getIsbn());
            }
        });

        assertEquals("978-1", books.get(0).getIsbn());
    }
}
```

복사하다 a와 b를 바꿔 썼다

`BookSortTest`를 실행하면 주문 화면은 통과, 재고 화면은 실패한다. 두 테스트에서 다른 곳은 한 줄뿐이다(발췌).

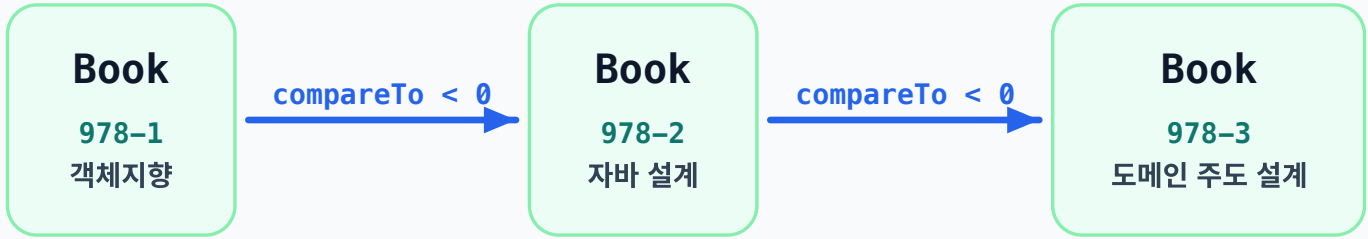
```
return a.getIsbn().compareTo(b.getIsbn());   // 주문 화면
return b.getIsbn().compareTo(a.getIsbn());   // 재고 화면: a와 b가 뒤바뀜
```

```
FAILED 재고 화면: 복사한 Comparator도 ISBN 순이어야 한다
       expected: <978-1> but was: <978-3>
```

- 주문 화면, 재고 화면, 배송 목록… 화면마다 여섯 줄이 복사된다. 컴파일러는 조용하고, 테스트만 잡았다.
- `import`의 `HashSet` · `Set` · `TreeSet`은 이 교시 뒤쪽 테스트에서 쓴다. 미리 넣어 두었다.

> 현업에서 본 장면: `Collections.sort`를 부를 때마다 익명 `Comparator`를 새로 만들던 서비스 코드.

타입 스스로 기본 순서를 안다



`a.compareTo(b)`가 **음수면 a가 앞**, 0이면 같은 자리, 양수면 a가 뒤다. 기준을 `Book` 안에 한 번만 두면 정렬하는 쪽은 아무것도 알려 주지 않아도 된다.

타입이 스스로 아는 기본 순서를 **자연 순서**라 하고, 그 약속이 `Comparable<Book>`의 `compareTo` 하나다.

> 비유: 서가의 책은 등에 붙은 청구기호가 자리를 정한다.

> 이것이 이펙티브 자바 아이템 14: Comparable을 구현할지 고려하라.

compareTo 최소 구현: ISBN 비교는 String에게 맡긴다

앞의 Book.java 에 `Comparable` 을 붙인다(클래스 선언과 `compareTo` 두 곳이 바뀌어 파일 전체).

```
public class Book implements Comparable<Book> {
    private String isbn;
    private String title;

    public Book(String isbn, String title) {
        this.isbn = isbn;
        this.title = title;
    }

    public String getIsbn() { return isbn; }
    public String getTitle() { return title; }

    @Override
    public int compareTo(Book other) {
        return isbn.compareTo(other.isbn);
    }
}
```

- `String` 이 이미 `Comparable` 이라 ISBN 비교를 그대로 넘긴다.
- `Comparable<Book>` 이라 매개변수가 `Object` 가 아니다. 캐스팅이 필요 없다.
- `final` , `equals` , `hashCode` 는 일부러 아직 두지 않았다. 연습문제에서 완성한다.

테스트: Collections.sort만 부르면 ISBN 순이다

`BookSortTest.java` 에서 익명 Comparator 테스트 두 개를 지우고, 그 자리(`books()` 아래)에 이 테스트를 넣는다.

`books()` 는 그대로 둔다(978-3, 978-1, 978-2 순서).

```
    @Test
    @DisplayName("Collections.sort만 부르면 ISBN 순으로 정렬된다")
    void sortsByIsbnNaturally() {
        List<Book> books = books();

        Collections.sort(books);

        assertEquals("978-1", books.get(0).getIsbn());
        assertEquals("978-2", books.get(1).getIsbn());
        assertEquals("978-3", books.get(2).getIsbn());
    }
```

> 통과. 화면마다 복사되던 익명 `Comparator` 가 `compareTo` 한 곳으로 모였다.

빼기로 비교하면 무엇이 위험할까? (퀴즈)

퀴즈

체력 순 비교를 `a.getHp() - b.getHp()`처럼 빼기로 쓰는 코드를 자주 본다. 빼기 비교만 떼어 보자.

```
static int byMinus(int a, int b) {
    return a - b;                // 음수면 a가 앞
}

byMinus(40, 60);                 // -20: 40이 앞. 맞다
byMinus(Integer.MAX_VALUE, -1); // ?
```

1. 양수 — `MAX_VALUE`가 뒤로 간다
2. 음수 — `MAX_VALUE`가 -1보다 앞으로 간다
3. 0 — 같은 자리
4. `ArithmeticException`이 난다

풀이: 정수 오버플로 — Integer.compare를 쓴다

정답 정답은 2번이다. 직접 실행해 보자.

CompareByMinusDemo.java

```java
public class CompareByMinusDemo {

    static int byMinus(int a, int b) {
        return a - b;
    }

    public static void main(String[] args) {
        System.out.println(byMinus(40, 60));
        System.out.println(byMinus(Integer.MAX_VALUE, -1));
        System.out.println(Integer.compare(Integer.MAX_VALUE, -1));
    }
}
```

실행 결과:

```
-20
-2147483648
1
```

- `MAX_VALUE - (-1)`은 넘쳐서 가장 작은 음수가 된다.
- `Integer.compare`는 빼지 않고 크기만 비교한다.
- 체력 순이라면 `Integer.compare(a.getHp(), b.getHp())`.

> 체력(0~150)에서는 지금 당장 터지지 않는다. 금액·ID·시각처럼 범위가 큰 값에 같은 습관을 옮겼을 때 터진다.

다른 기준은 Comparator로: 제목순

기본 순서는 하나만 정한다. 화면마다 다른 순서는 밖에서 넘긴다. `BookSortTest.java` 에 이어서 클래스 안에 추가한다.

```
    @Test
    @DisplayName("제목순은 Comparator로 따로 넘긴다")
    void sortsByTitleWithComparator() {
        List<Book> books = books();

        Collections.sort(books, Comparator.comparing(Book::getTitle));

        assertEquals("객체지향", books.get(0).getTitle());
        assertEquals("도메인 주도 설계", books.get(1).getTitle());
        assertEquals("자바 설계", books.get(2).getTitle());
    }
```

- `Comparator.comparing(Book::getTitle)` 은 "각 책의 `getTitle()` 값으로 비교하라"는 뜻이다. 문법은 Day 2에서.
- 「정렬할 때마다 익명 Comparator를 쓰게 된다」의 익명 클래스 여섯 줄이 한 줄이 됐고, `a` · `b` 를 바꿔 쓸 자리도 없다.

compareTo가 0이면 equals도 true여야 한다

지금 `Book`에는 `compareTo`만 있고 `equals`가 없다. 같은 책 두 권을 두 집합에 넣어 본다. `BookSortTest.java`에 이어서 클래스 안에 추가한다.

```
    @Test
    @DisplayName("equals 없이 compareTo만 있으면 HashSet은 2권, TreeSet은 1권으로 센다")
    void hashSetAndTreeSetDisagree() {
        Book first = new Book("978-1", "객체지향");
        Book second = new Book("978-1", "객체지향");

        Set<Book> hashSet = new HashSet<>();
        hashSet.add(first);
        hashSet.add(second);
        Set<Book> treeSet = new TreeSet<>();
        treeSet.add(first);
        treeSet.add(second);

        assertEquals(2, hashSet.size());
        assertEquals(1, treeSet.size());
    }
```

- 통과한다. `HashSet`은 `equals`로 2권, `TreeSet`은 `compareTo`로 1권이라 센다. 재고 수가 화면마다 달라지는 버그가 된다.
- 규칙: `compareTo`가 0인 두 객체는 `equals`도 true가 되게 같은 필드로 판단한다.

JDK의 BigDecimal도 이 규칙을 따르지 않는다

`1.0`과 `1.00`은 `compareTo`로는 같고 `equals`로는 다르다(자릿수까지 본다).

BigDecimalSetDemo.java

```
import java.math.BigDecimal;
import java.util.HashSet;
import java.util.Set;
import java.util.TreeSet;

public class BigDecimalSetDemo {

    public static void main(String[] args) {
        Set<BigDecimal> hashSet = new HashSet<>();
        hashSet.add(new BigDecimal("1.0"));
        hashSet.add(new BigDecimal("1.00"));

        Set<BigDecimal> treeSet = new TreeSet<>();
        treeSet.add(new BigDecimal("1.0"));
        treeSet.add(new BigDecimal("1.00"));

        System.out.println(hashSet.size());
        System.out.println(treeSet.size());
    }
}
```

실행 결과:

```
2
1
```

- `Book`과 같은 모양이다. 같은 금액 두 개가 `HashSet`에는 2개, `TreeSet`에는 1개.
- 금액을 다루는 Day 2에서 다시 만난다.

연습문제 시작 코드: 불변과 기본 순서는 채워 두었다

앞의 Book.java 를 이 시작 코드로 맞춘다(클래스와 두 필드에 `final` 추가). 4교시 불변 규칙과 이번 교시 `compareTo` 까지 들어 있다.

```
public final class Book implements Comparable<Book> {
    private final String isbn;
    private final String title;

    public Book(String isbn, String title) {
        this.isbn = isbn;
        this.title = title;
    }

    public String getIsbn() { return isbn; }
    public String getTitle() { return title; }

    @Override
    public int compareTo(Book other) {
        return isbn.compareTo(other.isbn);
    }

    // equals, hashCode: 실습에서 추가
}
```

- `final class` 라 하위 클래스가 순서나 같음을 바꿔 놓을 수 없다. setter는 없다.

연습문제 테스트: BookTest (1/2)

새 파일 `BookTest.java`. 채점 기준인 테스트 4건이다. 두 장을 이어서 붙여 넣는다.

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.TreeSet;

import static org.junit.jupiter.api.Assertions.assertEquals;

class BookTest {

    @Test
    @DisplayName("ISBN 순으로 정렬된다")
    void sortsByIsbn() {
        List<Book> books = new ArrayList<>();
        books.add(new Book("978-3", "도메인 주도 설계"));
        books.add(new Book("978-1", "객체지향"));
        books.add(new Book("978-2", "자바 설계"));

        Collections.sort(books);

        assertEquals("978-1", books.get(0).getIsbn());
        assertEquals("978-2", books.get(1).getIsbn());
        assertEquals("978-3", books.get(2).getIsbn());
    }
```

```
    @Test
    @DisplayName("ISBN이 같으면 같은 책이다")
    void sameIsbnMeansSameBook() {
        Book book = new Book("978-1", "객체지향");
        Book reprint = new Book("978-1", "객체지향");

        assertEquals(book, reprint);
    }

    @Test
    @DisplayName("HashSet에서 같은 ISBN은 하나로 센다")
    void hashSetCountsSameIsbnOnce() {
        Set<Book> books = new HashSet<>();

        books.add(new Book("978-1", "객체지향"));
        books.add(new Book("978-1", "객체지향"));

        assertEquals(1, books.size());
    }
```

연습문제 테스트: BookTest (2/2)

BookTest.java (이어서) — 마지막 테스트와 클래스를 닫는 괄호.

```
    @Test
    @DisplayName("TreeSet과 HashSet의 크기가 같다")
    void treeSetAndHashSetAgree() {
        List<Book> books = new ArrayList<>();
        books.add(new Book("978-1", "객체지향"));
        books.add(new Book("978-1", "객체지향"));
        books.add(new Book("978-2", "자바 설계"));

        Set<Book> hashSet = new HashSet<>(books);
        Set<Book> treeSet = new TreeSet<>(books);

        assertEquals(hashSet.size(), treeSet.size());
    }
}
```

- 같은 책 두 권 + 다른 책 한 권. `TreeSet`은 `compareTo`로 2권을 센다. `HashSet`도 2권이어야 한다.
- 지금 실행하면 「ISBN 순으로 정렬된다」만 통과하고 3건이 실패한다.

Day 1 연습문제: Book 값 객체를 완성한다

실습 10분 · 시작: `Book.java` (방금 맞춘 시작 코드) · 채점: `BookTest.java`

`BookTest.java` 4건 중 「ISBN 순으로 정렬된다」는 이미 통과하고, 3건이 실패한다.

실패하는 테스트	지금 메시지	고칠 것 (기준은 ISBN)
ISBN이 같으면 같은 책이다	`expected: <Book@…> but was: <Book@…>`	`equals` (7교시 `Position` 모양)
HashSet에서 같은 ISBN은 하나로 센다	`expected: <1> but was: <2>`	`hashCode`
TreeSet과 HashSet의 크기가 같다	`expected: <3> but was: <2>`	위 둘을 `compareTo`와 같은 필드로

실행: `BookTest` 전체 → 4건 통과. `equals`만 하고 `hashCode`를 빼먹으면 표의 HashSet·TreeSet 두 건이 그대로 남는다.

왜: Day 2의 `LineItem`이 이 `Book`을 그대로 담는다. 같은 책을 같다고 알아야 한다.

다 고치면 `BookSortTest`의 `hashSetAndTreeSetDisagree`는 실패한다(HashSet도 1권). equals 없는 Book을 보인 시연이었으니 그 테스트는 지운다.

연습문제 풀이: equals와 hashCode — ISBN 하나로

`Book.java` 최종 형태(바뀐 곳이 둘이라 파일 전체). Day 2가 이 파일을 그대로 쓴다.

```
import java.util.Objects;

public final class Book implements Comparable<Book> {
    private final String isbn;
    private final String title;

    public Book(String isbn, String title) {
        this.isbn = isbn;
        this.title = title;
    }

    public String getIsbn() { return isbn; }
    public String getTitle() { return title; }

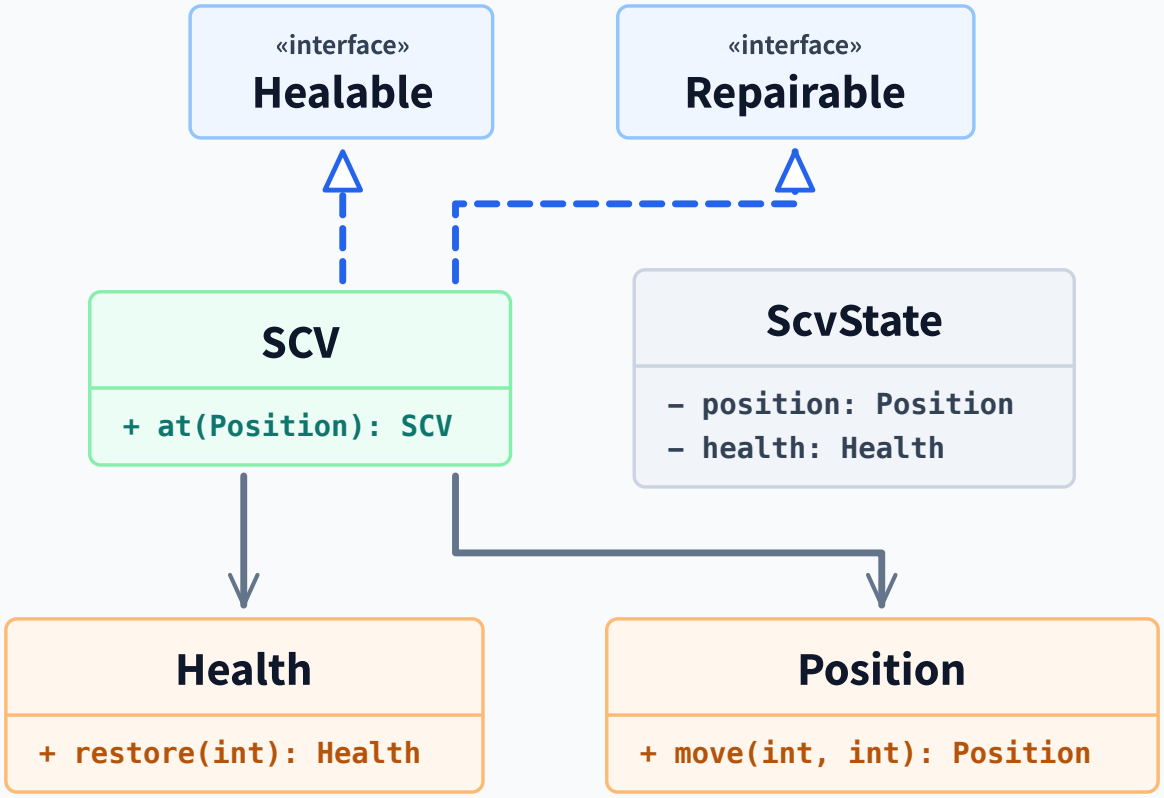
    @Override
    public int compareTo(Book other) {
        return isbn.compareTo(other.isbn);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) {
            return true;
        }
        if (!(o instanceof Book)) {
            return false;
        }
        Book other = (Book) o;
        return Objects.equals(isbn, other.isbn);
    }

    @Override
    public int hashCode() {
        return Objects.hash(isbn);
    }
}
```

> `BookTest` 4건 통과. 같음과 순서 모두 ISBN.

복습: 오늘 SCV가 거쳐 온 모양



역할은 인터페이스로, 체력과 위치는 불변 값 객체로 들고, 상태 기록 `ScvState`는 그 값들을 담는다.

복습: 문제 → 처방 → 아이템

교시	겪은 문제	처방	아이템
1	SCV는 생체인가 기계인가	역할은 인터페이스로	20
2	체력 계산식이 다섯 군데에	상속 대신 컴포지션(`Health`)	18
3	체력 1인 SCV를 시험할 수 없다	의존 객체 주입, 정적 팩토리	5, 1
4	집결지 순간이동 버그	방어적 복사, 불변 객체	50, 17
5	숫자 일곱 개의 의미	빌더	2
6	원래 예외가 사라졌다	try-with-resources	9
7	같은 좌표인데 못 찾는다	equals와 hashCode	10, 11
8	도서 목록 정렬	Comparable, 다른 기준은 Comparator	14

점검 질문 세 개

1. `Book`에는 `Comparable`을 구현하고 제목순은 `Comparator`로 넘겼다. 우리 프로젝트의 어떤 타입에 "기본 순서"가 있고, 어떤 순서는 화면마다 다른가?
2. 값 객체 체크리스트 — `final` 클래스와 `private final` 필드, `equals`, `hashCode`, 같은 필드로 판단하는 `compareTo` — 를 `Book` 코드에서 하나씩 짚을 수 있는가?
3. 복습 표 8개 문제 중 지난달 우리 코드에서 본 것 하나를 고르고, 내일 출근해서 바꿀 한 줄을 말해 보자.

내일 예고: 실수를 컴파일러가 잡게 한다

오늘은 실수를 **테스트**가 잡았다(체력 60, 순간이동, 사라진 예외, 못 찾는 좌표). 내일은 같은 종류의 실수가 **컴파일 단계**에서 막히게 하고, Java 8 문법을 원칙 있게 쓴다.

내일 겪을 문제	다룰 것
목록에서 꺼낼 때 터지는 `ClassCastException`	제네릭, 와일드카드
콜백마다 새 인터페이스, `forEach` 안의 `add`	람다, 표준 함수형 인터페이스, 스트림
`null`을 돌려주는 조회	빈 컬렉션, `Optional`
1원이 어긋난 정산, 동시에 담긴 주문	`BigDecimal`, 동기화

오늘 완성한 `Book`은 내일 주문 항목 `LineItem`이 그대로 이어받는다.